

ĐẠI HỌC BÁCH KHOA HÀ NỘI
TRƯỜNG CÔNG NGHỆ THÔNG TIN VÀ TRUYỀN THÔNG



ĐỒ ÁN MÔN HỌC

TÌM HIỂU VÀ NGHIÊN CỨU XÂY DỰNG IDS ĐỂ PHÁT HIỆN CÁC DẠNG TẤN CÔNG RPL

Sinh viên: Trần Hoàng Long - 20251057M
Hoàng Công Phát - 20251056M
Trần Mạnh Dũng - 20251195M

Ngành: Kỹ thuật máy tính

Học phần: An toàn thông tin trong IoT

Giảng viên hướng dẫn: Trần Quang Đức _____

Hà Nội, 04/05/2026.

LỜI NÓI ĐẦU

Sự bùng nổ của *Internet of Things* (IoT) trong những năm gần đây đã tạo ra một hạ tầng kết nối khổng lồ với hàng tỷ thiết bị nhúng hoạt động trong các môi trường đa dạng từ nhà thông minh, y tế, đến hệ thống công nghiệp và thành phố thông minh. Nền tảng định tuyến cho các mạng IoT hạn chế tài nguyên — giao thức **RPL** (RFC 6550) — đóng vai trò không thể thiếu trong việc duy trì liên lạc giữa các nút cảm biến. Tuy nhiên, thiết kế nhẹ nhàng của RPL cũng mang lại nhiều lỗ hổng bảo mật nghiêm trọng mà các biện pháp mặc định của giao thức chưa đủ sức bảo vệ.

Đồ án môn học này được thực hiện với mục tiêu tìm hiểu, phân tích bốn dạng tấn công tiêu biểu nhằm vào RPL — DIS Flooding, Blackhole, Decreased Rank và VeRA/DODAG Version Number Attack — đồng thời nghiên cứu và đề xuất kiến trúc **Hệ thống Phát hiện Xâm nhập** (IDS) có khả năng phát hiện các mối đe dọa này trong môi trường mô phỏng Contiki-NG/Cooja.

Trong quá trình thực hiện đồ án, chúng tôi xin gửi lời cảm ơn chân thành đến thầy **Trần Quang Đức**, người đã tận tình hướng dẫn, góp ý và động viên chúng tôi hoàn thành công trình này. Chúng tôi cũng xin cảm ơn các thầy cô trong **Trường Công nghệ Thông tin và Truyền thông** đã trang bị cho chúng tôi nền tảng kiến thức vững chắc trong suốt quá trình học tập.

Mặc dù đã cố gắng hết sức, đồ án chắc chắn còn nhiều thiếu sót. Chúng tôi rất mong nhận được sự góp ý của thầy cô và các bạn để công trình được hoàn thiện hơn.

Hà Nội, 04/05/2026.

Nhóm sinh viên thực hiện

**Trần Hoàng Long
Hoàng Công Phát
Trần Mạnh Dũng**

LỜI CAM ĐOAN

Chúng tôi xin cam đoan đồ án môn học **“Tìm hiểu và nghiên cứu xây dựng IDS để phát hiện các dạng tấn công RPL”** là công trình nghiên cứu của bản thân nhóm dưới sự hướng dẫn của thầy **Trần Quang Đức**.

Các nội dung nghiên cứu và kết quả trong đồ án này là trung thực và chưa từng được công bố dưới bất kỳ hình thức nào. Những số liệu, bảng biểu phục vụ cho việc phân tích, nhận xét và đánh giá được thu thập từ các nguồn tài liệu khác nhau đều có trích dẫn rõ ràng và ghi chú nguồn gốc cụ thể.

Nếu phát hiện có bất kỳ sự gian lận nào, chúng tôi xin hoàn toàn chịu trách nhiệm về nội dung đồ án của nhóm.

Hà Nội, 04/05/2026.

Nhóm sinh viên thực hiện

**Trần Hoàng Long
Hoàng Công Phát
Trần Mạnh Dũng**

MỤC LỤC

LỜI NÓI ĐẦU	3
LỜI CAM ĐOAN	5
DANH MỤC CHỮ VIẾT TẮT	11
DANH MỤC HÌNH ẢNH	13
DANH MỤC BẢNG BIỂU	14
TÓM TẮT	15
ABSTRACT	15
1 GIỚI THIỆU VÀ TỔNG QUAN	1
1.1 Bối cảnh và động lực nghiên cứu	1
1.2 Vấn đề nghiên cứu	2
1.3 Mục tiêu đề tài	3
1.4 Phạm vi nghiên cứu	3
1.5 Đóng góp của đề tài	4
1.6 Cấu trúc báo cáo	4
2 CƠ SỞ LÝ THUYẾT	6
2.1 Giao thức RPL	6
2.1.1 Tổng quan về RPL	6
2.1.2 Cấu trúc DODAG	6
2.1.3 Các bản tin điều khiển RPL	7
2.1.4 Bộ hẹn giờ Trickle	7
2.1.5 Objective Function	8
2.2 Phân loại tấn công RPL	8

2.2.1	Nhóm tấn công tài nguyên (Resources)	8
2.2.2	Nhóm tấn công tô-pô (Topology)	8
2.2.3	Nhóm tấn công lưu lượng (Traffic)	9
2.3	Tổng quan về IDS trong IoT/LLN	9
2.3.1	Khái niệm và phân loại IDS	9
2.3.2	Thách thức IDS trong LLN	10
2.4	Các chỉ số đánh giá hiệu năng	10
2.4.1	Chỉ số hiệu năng mạng	10
2.4.2	Chỉ số đánh giá IDS	11
2.5	Môi trường mô phỏng: Contiki-NG và Cooja	12
2.5.1	Contiki-NG	12
2.5.2	Cooja Simulator	12
2.5.3	Cấu hình mô phỏng chuẩn	13
2.6	Tóm tắt chương	13

3 PHÂN TÍCH CÁC DẠNG TẤN CÔNG RPL 14

3.1	DIS Flooding Attack	14
3.1.1	Cơ chế tấn công	14
3.1.2	Tác động đến hiệu năng mạng	14
3.1.3	Dấu hiệu nhận biết	15
3.1.4	Phòng chống	15
3.2	Blackhole Attack	15
3.2.1	Cơ chế tấn công	15
3.2.2	Tác động đến hiệu năng mạng	16
3.2.3	Dấu hiệu nhận biết	16
3.2.4	Phòng chống	16
3.3	Decreased Rank Attack	16
3.3.1	Cơ chế tấn công	16
3.3.2	Tác động đến hiệu năng mạng	17
3.3.3	Dấu hiệu nhận biết	17

3.3.4	Phòng chống	18
3.4	Tấn công phiên bản DODAG	18
3.4.1	Cơ chế tấn công	18
3.4.2	Tác động đến hiệu năng mạng	18
3.4.3	Dấu hiệu nhận biết	18
3.4.4	Phòng chống	19
3.5	So sánh tổng hợp bốn kiểu tấn công	19
3.6	Tóm tắt chương	20
4	THIẾT KẾ VÀ TRIỂN KHAI IDS PHÁT HIỆN TẤN CÔNG RPL	21
4.1	Phòng chống tấn công DIS flooding	21
4.2	Phòng chống tấn công phiên bản DAG	23
4.3	Thiết kế	23
4.3.1	Triển khai	24
4.4	Phòng chống tấn công blackhole	27
4.5	Thiết kế	27
4.5.1	Triển khai	28
4.6	Phòng chống tấn công hạ Rank (giả mạo rank thấp hơn)	34
4.6.1	Thiết kế	34
4.6.2	Triển khai	34
4.7	Tổng kết	38
5	THỰC NGHIỆM VÀ ĐÁNH GIÁ	39
5.1	Môi trường thực nghiệm	39
5.2	Các tiêu chí đánh giá	39
5.3	Kịch bản thực nghiệm	40
5.4	Kết quả	40
5.5	Phân tích tiêu thụ năng lượng	42
5.5.1	DIS Flooding	42
5.5.2	DAG Version Attack	43

5.5.3	Blackhole Attack	43
5.5.4	Decreased Rank Attack	43
5.6	Tổng kết	44
6	KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	45
6.1	Kết luận	45
6.2	Hướng phát triển	45
	TÀI LIỆU THAM KHẢO	47
	PHỤ LỤC	48
A	Mã nguồn IDS (Contiki-NG)	48
A.1	Module phát hiện DIS Flooding	48
A.2	Module phát hiện Blackhole	48
A.3	Module phát hiện Decreased Rank	48
A.4	Module phát hiện VeRA / Version Number Attack	49
B	Cấu hình mô phỏng Cooja	50
B.1	Tham số mô phỏng	50
B.2	Hướng dẫn chạy mô phỏng	50

DANH MỤC CHỮ VIẾT TẮT

Viết tắt	Tiếng Anh	Tiếng Việt
BR	Border Router	Bộ định tuyến biên
DAO	Destination Advertisement Object	Bản tin quảng bá đích
DAG	Directed Acyclic Graph	Đồ thị không chu trình có hướng
DIO	DODAG Information Object	Bản tin thông tin DODAG
DIS	DODAG Information Solicitation	Bản tin yêu cầu thông tin DODAG
DODAG	Destination Oriented Directed Acyclic Graph	Đồ thị acyclic có hướng tập trung vào đích
E2ED	End-to-End Delay	Trễ đầu cuối
FPR	False Positive Rate	Tỉ lệ cảnh báo nhầm
IDS	Intrusion Detection System	Hệ thống phát hiện xâm nhập
IETF	Internet Engineering Task Force	—
IoT	Internet of Things	Internet vạn vật
LLN	Low-power and Lossy Network	Mạng năng lượng thấp và mất mát cao
MOP	Mode of Operation	Chế độ hoạt động
OF	Objective Function	Hàm mục tiêu
PDR	Packet Delivery Ratio	Tỉ lệ phân phối gói tin thành công
RFC	Request for Comments	—
RPL	Routing Protocol for Low-Power and Lossy Networks	Giao thức định tuyến cho LLN
TPR	True Positive Rate	Tỉ lệ phát hiện đúng
VeRA	Version Number and Rank Authentication	Xác thực số phiên bản và rank
WSN	Wireless Sensor Network	Mạng cảm biến không dây

DANH MỤC HÌNH ẢNH

Hình 3.1	Cơ chế DIS Flooding Attack: kẻ tấn công A liên tục phát DIS, buộc các nút lân cận reset Trickle Timer và phát DIO	14
Hình 5.2	Tiêu thụ năng lượng DIS Flooding: CPU (trái) và Radio (phải) . .	42
Hình 5.3	Tiêu thụ năng lượng DAG Version Attack: CPU (trái) và Radio (phải)	43
Hình 5.4	Tiêu thụ năng lượng Blackhole Attack: CPU (trái) và Radio (phải)	44
Hình 5.5	Tiêu thụ năng lượng Decreased Rank Attack: CPU (trái) và Radio (phải)	44

DANH MỤC BẢNG BIỂU

Bảng 2.1	Ma trận nhầm lẫn của IDS	12
Bảng 2.2	Tham số cấu hình mô phỏng Cooja	13
Bảng 3.3	So sánh bốn kiểu tấn công RPL được nghiên cứu	20
Bảng 5.4	Kết quả đánh giá độ chính xác trong việc phát hiện tấn công . . .	41
Bảng 5.5	Tổng số gói tin (DIO / DIS / DAO) theo kịch bản	41
Bảng 5.6	Packet Delivery Ratio (PDR) - Chi tiết Upstream/Downstream . .	42
Bảng B.7	Tham số cấu hình chung trong Cooja	50

TÓM TẮT

Giao thức định tuyến **RPL** (IPv6 Routing Protocol for Low-Power and Lossy Networks, RFC 6550) là chuẩn định tuyến chủ đạo trong các mạng IoT năng lượng thấp (LLN). Tuy nhiên, thiết kế nhẹ nhàng của RPL tạo ra bề mặt tấn công rộng mà các cơ chế bảo mật mặc định của giao thức chưa bảo vệ được đầy đủ, đặc biệt khi đối mặt với các nút nội bộ bị xâm phạm.

Đồ án này trình bày quá trình nghiên cứu, phân tích và xây dựng **Hệ thống Phát hiện Xâm nhập** (IDS) cho mạng RPL. Bốn dạng tấn công tiêu biểu được tập trung nghiên cứu gồm: *DIS Flooding Attack*, *Blackhole Attack*, *Decreased Rank Attack* và *VeRA/DODAG Version Number Attack*. Với mỗi dạng tấn công, báo cáo phân tích cơ chế hoạt động, tác động đến hiệu năng mạng và phương pháp phát hiện tương ứng.

Kiến trúc IDS đề xuất được tích hợp trực tiếp vào ngăn xếp Contiki-NG, không yêu cầu sửa đổi nhân giao thức RPL. Hệ thống được kiểm thử trên bộ mô phỏng Cooja với nhiều kịch bản khác nhau. Kết quả thực nghiệm cho thấy IDS đề xuất đạt tỉ lệ phát hiện cao ($TPR \geq 90\%$) với tỉ lệ cảnh báo nhầm thấp ($FPR \leq 10\%$), đồng thời duy trì overhead tài nguyên ở mức chấp nhận được trên các nút cảm biến hạn chế.

Từ khóa: RPL, IDS, IoT, LLN, DIS Flooding, Blackhole, Decreased Rank, VeRA, Contiki-NG, Cooja, phát hiện xâm nhập.

ABSTRACT

The **RPL** protocol (IPv6 Routing Protocol for Low-Power and Lossy Networks, RFC 6550) is the de-facto routing standard for IoT and LLN environments. However, its lightweight design introduces a wide attack surface that default RPL security mechanisms cannot adequately protect against, especially when facing compromised internal nodes.

This report presents the research, analysis, and design of an **Intrusion Detection System** (IDS) for RPL networks. Four representative attack types are studied in depth:

DIS Flooding Attack, Blackhole Attack, Decreased Rank Attack, and VeRA/DODAG Version Number Attack. For each attack, the report analyzes the mechanism, impact on network performance, and the corresponding detection method.

The proposed IDS architecture is integrated directly into the Contiki-NG stack without modifying the RPL protocol core. The system is evaluated on the Cooja Simulator under multiple scenarios. Experimental results show that the proposed IDS achieves high detection rates ($\text{TPR} \geq 90\%$) with low false positive rates ($\text{FPR} \leq 10\%$), while maintaining acceptable resource overhead on constrained sensor nodes.

Keywords: RPL, IDS, IoT, LLN, DIS Flooding, Blackhole, Decreased Rank, VeRA, Contiki-NG, Cooja, intrusion detection.

1 GIỚI THIỆU VÀ TỔNG QUAN

1.1 Bối cảnh và động lực nghiên cứu

Trong những thập kỷ gần đây, *Internet of Things* (IoT) đã trở thành một trong những xu hướng công nghệ phát triển nhanh nhất, kết nối hàng tỷ thiết bị thông minh trong các lĩnh vực từ nhà thông minh, y tế, nông nghiệp chính xác đến hệ thống công nghiệp và thành phố thông minh [1]. Theo dự báo của các tổ chức nghiên cứu công nghệ, số lượng thiết bị IoT được kết nối trên toàn cầu sẽ vượt mốc 30 tỷ thiết bị vào năm 2030, tạo nên một hạ tầng mạng lưới phức tạp và phân tán chưa từng có trong lịch sử.

Nền tảng vật lý của hạ tầng IoT chủ yếu dựa trên các *Mạng năng lượng thấp và mất mát cao* (Low-power and Lossy Networks – LLN). Đây là các mạng không dây bao gồm hàng nghìn nút cảm biến bị giới hạn nghiêm ngặt về bộ nhớ, khả năng xử lý và nguồn năng lượng pin [1]. Các nút trong LLN thường được triển khai trong những môi trường khắc nghiệt nơi việc thay thế pin là không khả thi, đòi hỏi giao thức mạng phải cực kỳ tiết kiệm tài nguyên.

Để giải quyết bài toán định tuyến hiệu quả trong môi trường ràng buộc này, Tổ chức Kỹ thuật Internet (IETF), thông qua nhóm làm việc ROLL (*Routing Over Low-power and Lossy Networks*), đã chuẩn hóa **RPL** (*IPv6 Routing Protocol for Low-Power and Lossy Networks*) theo RFC 6550 vào năm 2012 [2]. RPL nhanh chóng trở thành giao thức định tuyến tiêu chuẩn *de facto* cho IoT/LLN nhờ thiết kế nhẹ nhàng, hỗ trợ đa dạng mô hình truyền thông (nhiều-đến-một, một-đến-nhiều, điểm-đến-điểm) và khả năng tự tổ chức tô-pô thích ứng.

Tuy nhiên, chính sự đơn giản và phổ biến của RPL lại tạo ra bề mặt tấn công đáng lo ngại. Các nghiên cứu gần đây đã chỉ ra rằng RPL rất dễ bị tổn thương trước nhiều dạng tấn công khác nhau, gây ra các hậu quả nghiêm trọng như: tăng tiêu thụ năng lượng, định tuyến gói sai hướng, tắc nghẽn mạng và cô lập các nút hợp lệ [1]. Đặc biệt nguy hiểm là các cuộc tấn công từ *nút nội bộ bị xâm phạm* (compromised internal node) — loại tấn công mà ngay cả cơ chế mật mã tùy chọn của RPL cũng không bảo vệ được [3].

Trong bối cảnh đó, nhu cầu xây dựng **Hệ thống Phát hiện Xâm nhập** (Intrusion Detection System – IDS) chuyên biệt cho mạng RPL ngày càng trở nên cấp thiết, đặc biệt khi các ứng dụng IoT trong y tế, cơ sở hạ tầng trọng yếu và quân sự yêu cầu mức độ tin cậy và an toàn cao.

1.2 Vấn đề nghiên cứu

RPL xây dựng và duy trì một đồ thị acyclic có hướng tập trung vào đích (*Destination Oriented Directed Acyclic Graph* – DODAG) thông qua ba loại thông điệp điều khiển chính: DIO (DODAG Information Object), DIS (DODAG Information Solicitation) và DAO (Destination Advertisement Object). Thiết kế tin tưởng ngầm định vào tính trung thực của các nút tham gia mạng, điều này hoàn toàn có thể bị khai thác bởi các nút độc hại.

Theo phân loại trong [4], các tấn công vào RPL thuộc ba nhóm chính:

1. **Tấn công cạn kiệt tài nguyên (Resources):** Nút độc hại buộc các nút hợp lệ thực hiện những hành động không cần thiết, làm tăng tiêu thụ năng lượng, bộ nhớ và băng thông. Nhóm này bao gồm tấn công trực tiếp (direct) như DIS Flooding, và gián tiếp (indirect) như Version Number Attack.
2. **Tấn công phá vỡ tô-pô (Topology):** Nút độc hại làm méo cấu trúc DODAG, khiến mạng hội tụ về trạng thái không tối ưu (sub-optimization) hoặc cô lập một số nút (isolation). Nhóm này bao gồm Decreased Rank Attack và Blackhole Attack.
3. **Tấn công lưu lượng (Traffic):** Nút độc hại len lỏi vào mạng để nghe trộm (eavesdropping) hoặc giả mạo (misappropriation) lưu lượng dữ liệu hợp lệ.

Đồ án tập trung vào **bốn dạng tấn công** tiêu biểu và nguy hiểm nhất, có đầy đủ tài liệu nghiên cứu và có thể mô phỏng kiểm thử trên Cooja:

1. **DIS Flooding Attack** [*Resources – Direct*]: Kẻ tấn công liên tục phát tán bản tin DIS vào mạng, buộc tất cả các nút lân cận phải reset bộ hẹn giờ Trickle và phản hồi bằng bản tin DIO, dẫn đến bùng nổ lưu lượng điều khiển và cạn kiệt năng lượng pin [1].
2. **Blackhole Attack** [*Topology – Isolation*]: Nút tấn công quảng bá các thông số định tuyến hấp dẫn để thu hút lưu lượng về phía mình, sau đó lặng lẽ hủy toàn bộ gói tin thay vì chuyển tiếp, gây mất gói diện rộng và có thể cô lập các nhánh mạng khỏi nút gốc [5].
3. **Decreased Rank Attack** [*Topology – Sub-optimization*]: Nút tấn công quảng bá giá trị Rank thấp hơn thực tế để lôi kéo các nút lân cận chọn nó làm nút cha ưu tiên, từ đó kiểm soát luồng dữ liệu và làm méo cấu trúc DODAG [?].
4. **VeRA / DODAG Version Number Attack** [*Resources – Indirect*]: Kẻ tấn công tăng số phiên bản DODAG bất hợp pháp, buộc toàn bộ mạng tái cấu trúc định tuyến

toàn cục (global repair), tiêu tốn lượng lớn năng lượng và băng thông điều khiển [3].

Mặc dù đã có các nghiên cứu đề xuất giải pháp phòng chống từng dạng tấn công riêng lẻ, việc xây dựng một IDS thống nhất có khả năng phát hiện đồng thời nhiều kiểu tấn công trên thiết bị cực kỳ hạn chế tài nguyên vẫn là một thách thức mở.

1.3 Mục tiêu đề tài

Đề án hướng tới năm mục tiêu cụ thể sau:

- M1. Nghiên cứu lý thuyết:** Nắm vững nguyên lý hoạt động của RPL (DODAG, Trickle Timer, Objective Function, DIO/DIS/DAO), phân loại tấn công và các khái niệm nền tảng của IDS trong môi trường IoT/LLN.
- M2. Phân tích tấn công:** Hiểu sâu cơ chế, tác động đến hiệu năng mạng và dấu hiệu nhận biết của 4 dạng tấn công RPL được chọn thông qua tài liệu và mô phỏng thực nghiệm.
- M3. Thiết kế IDS:** Đề xuất kiến trúc IDS phù hợp với ràng buộc tài nguyên của LLN, tích hợp module phát hiện độc lập cho từng kiểu tấn công mà không cần sửa đổi nhân RPL.
- M4. Triển khai và kiểm thử:** Cài đặt IDS trên Contiki-NG và mô phỏng trên Cooja theo ba trạng thái: baseline (bình thường), đang bị tấn công và có IDS bảo vệ.
- M5. Đánh giá định lượng:** So sánh các chỉ số PDR, End-to-End Delay, Control Overhead, Power Consumption, Detection Rate (TPR) và False Positive Rate (FPR) trước và sau khi triển khai IDS.

1.4 Phạm vi nghiên cứu

Để đảm bảo tính khả thi trong khuôn khổ đề án môn học, phạm vi được giới hạn cụ thể như sau:

- **Giao thức:** Chỉ nghiên cứu RPL (RFC 6550). Không bao gồm các giao thức LLN khác như OSPF, IS-IS hay AODV.
- **Mô hình kẻ tấn công:** Giả định tấn công từ *nút nội bộ bị xâm phạm* (compromised internal node). Đây là mô hình nguy hiểm nhất vì kẻ tấn công đã vượt qua cơ chế mật mã mặc định của RPL, có khả năng gửi bản tin hợp lệ về cú pháp nhưng mang nội dung độc hại.

- **Môi trường:** Mô phỏng trên Contiki-NG và Cooja Simulator. Không triển khai trên phần cứng thực tế.
- **Phương pháp IDS:** Tập trung vào anomaly-based và threshold-based detection. Không đi sâu vào học máy hoặc học sâu.
- **Quy mô mô phỏng:** Mạng tĩnh từ 10 đến 30 nút, tỉ lệ nút tấn công từ 0% đến 50%, thời gian mô phỏng 300 giây mỗi kịch bản.

1.5 Đóng góp của đề tài

So với các nghiên cứu liên quan, đề án có bốn đóng góp chính:

1. **Khung phân tích thống nhất:** Tổng hợp và phân tích chi tiết 4 dạng tấn công RPL trong một khung đánh giá nhất quán với cùng bộ chỉ số hiệu năng, cho phép so sánh trực tiếp mức độ nguy hại của từng kiểu tấn công.
2. **Kiến trúc IDS tích hợp:** Đề xuất IDS có khả năng phát hiện đồng thời nhiều kiểu tấn công mà không yêu cầu sửa đổi nhân RPL, đảm bảo tương thích với các triển khai Contiki-NG hiện có.
3. **Bộ kịch bản kiểm thử chuẩn hóa:** Cung cấp bộ kịch bản Cooja có thể tái lập (reproducible), giúp cộng đồng nghiên cứu dễ dàng so sánh và mở rộng.
4. **Phân tích đánh đổi tài nguyên:** Làm rõ sự đánh đổi (trade-off) giữa độ chính xác phát hiện và chi phí tài nguyên phát sinh bởi IDS trên các nút cảm biến hạn chế.

1.6 Cấu trúc báo cáo

Báo cáo được tổ chức thành 6 chương như sau:

- **Chương 1 – Giới thiệu và tổng quan** (chương hiện tại): Trình bày bối cảnh, vấn đề nghiên cứu, mục tiêu, phạm vi và cấu trúc báo cáo.
- **Chương 2 – Cơ sở lý thuyết:** Giới thiệu chi tiết giao thức RPL, phân loại tấn công theo taxonomy, tổng quan về IDS trong IoT, các chỉ số đánh giá hiệu năng và môi trường Contiki-NG/Cooja.
- **Chương 3 – Phân tích các dạng tấn công RPL:** Mô tả chi tiết cơ chế, tác động và dấu hiệu nhận biết của 4 kiểu tấn công, kèm kết quả mô phỏng so sánh với baseline.
- **Chương 4 – Thiết kế IDS phát hiện tấn công RPL:** Khảo sát related work, đề xuất kiến trúc IDS và trình bày các module phát hiện/phòng chống tương ứng.

- **Chương 5 – Demo và kiểm thử:** Trình bày môi trường thực nghiệm, kịch bản kiểm thử, kết quả đo lường (PDR, E2ED, overhead, power) và đánh giá IDS (TPR, FPR).
- **Chương 6 – Kết luận và hướng phát triển:** Tóm tắt kết quả, hạn chế và các hướng nghiên cứu tiếp theo.

Ngoài ra, báo cáo bao gồm **Phụ lục** với mã nguồn IDS (Contiki-NG), script mô phỏng Cooja và danh mục chữ viết tắt.

2 CƠ SỞ LÝ THUYẾT

2.1 Giao thức RPL

2.1.1 Tổng quan về RPL

Giao thức **RPL** (*IPv6 Routing Protocol for Low-Power and Lossy Networks*) được IETF chuẩn hóa theo RFC 6550 năm 2012, là giao thức định tuyến chuẩn *de facto* cho các mạng IoT năng lượng thấp và mất mát cao (LLN) [2]. RPL hoạt động ở tầng mạng, theo hướng tiếp cận định tuyến vector khoảng cách (distance vector), và tổ chức mạng thành cấu trúc đồ thị acyclic có hướng tập trung vào đích (DODAG).

RPL được thiết kế để đáp ứng bốn mô hình truyền thông chính trong IoT:

- **Nhiều-đến-một (Many-to-One):** Các nút cảm biến gửi dữ liệu về nút gốc. Đây là mô hình phổ biến nhất trong thu thập dữ liệu cảm biến.
- **Một-đến-nhiều (One-to-Many):** Nút gốc phát lệnh hoặc cấu hình xuống các nút cảm biến (downward routing).
- **Điểm-đến-điểm (Point-to-Point):** Hai nút bất kỳ trao đổi trực tiếp thông qua nút gốc.
- **Đa điểm-đến-đa điểm (Multipoint-to-Multipoint):** Hỗ trợ qua các cơ chế mở rộng.

2.1.2 Cấu trúc DODAG

DODAG (*Destination Oriented Directed Acyclic Graph*) là cấu trúc tô-pô cốt lõi mà RPL xây dựng và duy trì. Mỗi DODAG được đặc trưng bởi:

- **DODAG Root (Nút gốc / Border Router – BR):** Nút đặc biệt kết nối LLN với mạng IP bên ngoài. Mọi luồng upward đều hướng về nút này.
- **DODAGID:** Địa chỉ IPv6 toàn cục duy nhất định danh một DODAG.
- **DODAGVersionNumber:** Bộ đếm 8-bit, tăng mỗi khi nút gốc khởi tạo tái cấu trúc tô-pô toàn cục (global repair).
- **Rank:** Giá trị số nguyên không âm biểu diễn vị trí tương đối của nút so với nút gốc. Nút gốc có Rank = 1; các nút càng xa gốc có Rank càng lớn. Rank đóng vai trò quan trọng trong việc ngăn chặn vòng lặp định tuyến.

Mỗi nút không phải gốc phải chọn một **nút cha ưu tiên** (*preferred parent*) trong số các nút có Rank nhỏ hơn. Tập hợp các nút cha khả dĩ tạo thành *parent set*. Luồng dữ liệu upward đi từ nút lá (leaf node) qua các nút cha lên tới nút gốc theo cấu trúc cây phân cấp.

2.1.3 Các bản tin điều khiển RPL

RPL sử dụng bốn loại bản tin điều khiển ICMPv6 để xây dựng và duy trì DODAG [1]:

2.1.3.1 DODAG Information Object (DIO) DIO là bản tin quảng bá định tuyến quan trọng nhất. Nút gốc phát DIO multicast định kỳ; các nút trung gian sau khi nhận DIO sẽ cập nhật thông tin và chuyển tiếp DIO xuống các nút con. DIO chứa các trường chính: DODAGID, DODAGVersionNumber, Rank của người gửi, Mode of Operation (MOP) và thông tin Objective Function.

2.1.3.2 DODAG Information Solicitation (DIS) DIS là bản tin yêu cầu thông tin DODAG, được một nút gửi khi muốn gia nhập hoặc tái gia nhập mạng. Khi nhận DIS, các nút hàng xóm đang là thành viên DODAG sẽ phản hồi bằng DIO và **reset bộ hẹn giờ Trickle**, dẫn đến phát DIO ngay lập tức thay vì chờ hết chu kỳ hiện tại.

2.1.3.3 Destination Advertisement Object (DAO) DAO được gửi theo hướng upward để thông báo khả năng tiếp cận (*reachability*) của các nút xuôi dòng. Nút gốc dùng thông tin DAO để xây dựng bảng định tuyến downward, cho phép gửi dữ liệu từ gốc xuống bất kỳ nút nào trong DODAG.

2.1.3.4 DAO Acknowledgment (DAO-ACK) DAO-ACK xác nhận việc tiếp nhận và xử lý thành công một bản tin DAO, đảm bảo độ tin cậy trong việc thiết lập đường định tuyến downward.

2.1.4 Bộ hẹn giờ Trickle

Trickle Timer là cơ chế điều tiết tần suất phát bản tin DIO, cho phép RPL thích ứng với điều kiện mạng [1]. Hoạt động theo nguyên tắc:

1. Mỗi chu kỳ I bắt đầu với khoảng thời gian I_{min} và tăng gấp đôi sau mỗi chu kỳ ổn định, tới giới hạn I_{max} .
2. Trong mỗi chu kỳ, nút đếm số lần nghe thấy DIO nhất quán (counter c). Nếu $c < k$ (redundancy constant), nút phát DIO tại thời điểm ngẫu nhiên trong $[I/2, I]$.
3. Khi phát hiện không nhất quán (nhận DIS, thay đổi parent, routing loop...), Trickle

Timer bị **reset** về I_{min} , kích hoạt phát DIO ngay lập tức để sửa chữa nhanh.

Cơ chế này giảm thiểu overhead điều khiển khi mạng ổn định, nhưng cũng là điểm yếu bị khai thác bởi DIS Flooding Attack.

2.1.5 Objective Function

Objective Function (OF) là tập quy tắc xác định cách tính Rank từ các routing metrics và cách chọn preferred parent [?]. RPL chuẩn hóa hai OF:

- **OF0** (RFC 6552): Tối thiểu hóa số hop đến nút gốc. Rank được tính bằng cách cộng một giá trị không đổi (rank-increase) vào Rank của preferred parent. OF0 luôn chọn nút có Rank thấp nhất làm parent mà không có cơ chế chống dao động (anti-churn), khiến nó dễ bị ảnh hưởng bởi Decreased Rank Attack.
- **MRHOF** (RFC 6719): Sử dụng metric ETX (Expected Transmission Count) để chọn đường có chất lượng link tốt nhất. MRHOF tích hợp *hysteresis* — nút chỉ thay đổi parent khi ETX mới tốt hơn ngưỡng Δ_{ETX} so với parent hiện tại, giúp giảm parent churn nhưng vẫn có thể bị tấn công Rank nếu Δ_{ETX} đủ lớn.

2.2 Phân loại tấn công RPL

Theo khung phân loại được trình bày trong [4], các tấn công vào RPL được tổ chức thành ba nhóm chính, phản ánh ba chiều tác động khác nhau lên mạng:

2.2.1 Nhóm tấn công tài nguyên (Resources)

Mục tiêu là làm cạn kiệt năng lượng, bộ nhớ hoặc băng thông của các nút hợp lệ, từ đó rút ngắn tuổi thọ mạng. Nhóm này chia thành hai loại:

- **Tấn công trực tiếp (Direct)**: Nút độc hại trực tiếp tạo ra lưu lượng quá mức. Ví dụ điển hình: *DIS Flooding Attack* — phát tán DIS liên tục khiến mạng bùng nổ DIO.
- **Tấn công gián tiếp (Indirect)**: Nút độc hại kích thích các nút hợp lệ tạo ra lưu lượng dư thừa. Ví dụ: *Version Number Attack* — tăng DODAGVersionNumber bất hợp pháp buộc toàn mạng tái cấu trúc.

2.2.2 Nhóm tấn công tô-pô (Topology)

Mục tiêu là phá vỡ cấu trúc DODAG, gây định tuyến sai hoặc cô lập nút. Chia thành:

- **Sub-optimization:** Mạng hội tụ về trạng thái không tối ưu, giảm hiệu năng nhưng không cắt đứt kết nối hoàn toàn. Ví dụ: *Decreased Rank Attack*, *Routing Table Falsification*.
- **Isolation:** Một hoặc nhiều nút bị cô lập khỏi DODAG. Ví dụ: *Blackhole Attack*, *Sinkhole Attack*, *Wormhole Attack*.

2.2.3 Nhóm tấn công lưu lượng (Traffic)

Mục tiêu là khai thác hoặc giả mạo dữ liệu lưu thông trong mạng. Chia thành:

- **Eavesdropping (Passive):** Nghe trộm lưu lượng mà không can thiệp. Ví dụ: *Sniffing*, *Traffic Analysis*.
- **Misappropriation (Active):** Chiếm đoạt và thay đổi lưu lượng. Ví dụ: *Decreased Rank Attack*, *Identity Attack*.

2.3 Tổng quan về IDS trong IoT/LLN

2.3.1 Khái niệm và phân loại IDS

Hệ thống Phát hiện Xâm nhập (IDS) là tập hợp các công cụ và phương pháp giám sát hoạt động mạng hoặc hệ thống, phát hiện các hành vi bất thường hoặc vi phạm chính sách bảo mật. Trong môi trường IoT/LLN, IDS phải đối mặt với thách thức đặc biệt: tài nguyên tính toán cực kỳ hạn chế, môi trường truyền thông không đáng tin cậy và topology mạng thay đổi động.

Các IDS cho RPL/LLN được phân loại theo hai chiều chính:

2.3.1.1 Theo phương pháp phát hiện:

- **Signature-based:** So sánh hành vi quan sát được với cơ sở dữ liệu mẫu tấn công đã biết. Độ chính xác cao, FPR thấp nhưng không phát hiện được tấn công mới (zero-day).
- **Anomaly-based:** Xây dựng mô hình hành vi bình thường và phát hiện lệch chuẩn. Phát hiện được tấn công mới nhưng FPR thường cao hơn.
- **Threshold-based:** Dạng đặc biệt của anomaly-based, sử dụng ngưỡng cố định hoặc động cho các metrics cụ thể (tần suất DIS, PDR, Rank...). Nhẹ nhàng, phù hợp với LLN.
- **Hybrid:** Kết hợp nhiều phương pháp để tận dụng ưu điểm của từng loại [1].

2.3.1.2 Theo kiến trúc triển khai:

- **Centralized:** IDS tập trung tại nút gốc hoặc server ngoài. Xử lý mạnh nhưng tạo điểm đơn hỏng (single point of failure) và overhead truyền thông cao.
- **Distributed:** Mỗi nút chạy module IDS cục bộ, chia sẻ thông tin với hàng xóm. Khả năng mở rộng tốt, phù hợp LLN.
- **Hybrid:** Kết hợp giám sát cục bộ tại mỗi nút với phân tích tổng hợp tại nút gốc [1].

2.3.2 Thách thức IDS trong LLN

Thiết kế IDS cho RPL/LLN phải giải quyết các thách thức đặc thù:

1. **Ràng buộc tài nguyên:** Nút cảm biến thường chỉ có vài KB RAM, vài chục KB Flash và nguồn pin giới hạn. IDS không được tiêu tốn quá nhiều CPU cycles hay bộ nhớ.
2. **Môi trường truyền thông không tin cậy:** Packet loss tự nhiên cao (LLN) có thể bị nhầm là tấn công Blackhole, gây FPR cao.
3. **Tô-pô động:** Thay đổi parent hợp lệ (do di chuyển, pin yếu) có thể giống với dấu hiệu tấn công.
4. **Phân biệt tấn công và lỗi mạng:** Cần cơ chế phân biệt hành vi độc hại với lỗi giao thức thông thường.

2.4 Các chỉ số đánh giá hiệu năng

Để đánh giá toàn diện tác động của tấn công và hiệu quả của IDS, đề án sử dụng hai nhóm chỉ số sau:

2.4.1 Chỉ số hiệu năng mạng

2.4.1.1 *Packet Delivery Ratio (PDR)* PDR đo tỉ lệ gói dữ liệu được gửi thành công đến đích trên tổng số gói được phát đi:

$$PDR = \frac{N_{received}}{N_{sent}} \times 100\% \quad (2.1)$$

PDR là chỉ số phản ánh trực tiếp nhất mức độ ảnh hưởng của tấn công lên dịch vụ dữ liệu. Tấn công Blackhole và Decreased Rank làm giảm PDR đáng kể; tấn công DIS Flooding và VeRA gây suy giảm PDR gián tiếp qua overhead điều khiển.

2.4.1.2 End-to-End Delay (E2ED) E2ED là thời gian trung bình từ khi một gói dữ liệu được phát cho đến khi đến đích thành công:

$$E2ED = \frac{1}{N_{\text{received}}} \sum_{i=1}^{N_{\text{received}}} (t_{\text{receive},i} - t_{\text{send},i}) \quad (2.2)$$

E2ED tăng khi mạng bị tắc nghẽn (DIS Flooding) hoặc khi gói phải truyền qua đường dài hơn do tô-pô bị méo (Decreased Rank).

2.4.1.3 Control Overhead (CO) CO đo số lượng bản tin điều khiển RPL (DIO, DIS, DAO) phát sinh trong một khoảng thời gian quan sát. Tần công DIS Flooding và VeRA trực tiếp làm tăng CO lên nhiều lần so với baseline.

2.4.1.4 Power Consumption Tiêu thụ năng lượng trung bình của mỗi nút (mW hoặc mJ), đo bằng công cụ Powertrace trong Contiki-NG. Năng lượng tiêu thụ tăng khi nút phải xử lý nhiều bản tin điều khiển hơn hoặc phải tái cấu trúc tô-pô thường xuyên.

2.4.2 Chỉ số đánh giá IDS

2.4.2.1 True Positive Rate (TPR) – Detection Rate Tỷ lệ các cuộc tấn công thực sự được IDS phát hiện đúng:

$$TPR = \frac{TP}{TP + FN} \times 100\% \quad (2.3)$$

trong đó TP là số trường hợp tấn công phát hiện đúng, FN là số trường hợp tấn công bị bỏ sót.

2.4.2.2 False Positive Rate (FPR) Tỷ lệ các trường hợp bình thường bị IDS cảnh báo nhầm là tấn công:

$$FPR = \frac{FP}{FP + TN} \times 100\% \quad (2.4)$$

trong đó FP là số cảnh báo nhầm, TN là số trường hợp bình thường được phân loại đúng. FPR cao làm tăng overhead và gây phiền nhiễu vận hành.

2.4.2.3 Ma trận nhầm lẫn (Confusion Matrix) Tổng hợp kết quả phân loại của IDS qua bốn giá trị TP, TN, FP, FN như Bảng 2.1.

Bảng 2.1 Ma trận nhầm lẫn của IDS

	Thực tế: Tấn công	Thực tế: Bình thường
Dự đoán: Tấn công	TP	FP
Dự đoán: Bình thường	FN	TN

2.5 Môi trường mô phỏng: Contiki-NG và Cooja

2.5.1 Contiki-NG

Contiki-NG là hệ điều hành mã nguồn mở, nhẹ nhàng dành riêng cho các thiết bị IoT nhúng hạn chế tài nguyên. Contiki-NG là phiên bản kế thừa và được tái cấu trúc hoàn toàn từ Contiki OS, với các cải tiến về độ ổn định, bảo mật và khả năng kiểm thử. Một số đặc điểm nổi bật:

- Hỗ trợ đầy đủ ngăn xếp IPv6/6LoWPAN và RPL theo RFC 6550.
- Lập lịch phân tán dựa trên protothreads, tiêu thụ rất ít bộ nhớ (vài KB RAM).
- Công cụ **Powertrace** cho phép đo tiêu thụ năng lượng chi tiết theo từng module (CPU, radio Tx, radio Rx...).
- Tích hợp sẵn Cooja Simulator và hỗ trợ cross-compile cho nhiều phần cứng (Tmote Sky, Zolertia RE-Mote, nRF52840...).

2.5.2 Cooja Simulator

Cooja là bộ mô phỏng mạng cảm biến tích hợp trong Contiki-NG, cho phép mô phỏng hàng chục đến hàng trăm nút trên một máy tính thông thường [4]. Đặc điểm:

- **Mô phỏng mức bytecode:** Chạy trực tiếp bytecode của Contiki-NG, đảm bảo hành vi hoàn toàn giống phần cứng thực.
- **Mô hình radio linh hoạt:** Hỗ trợ Unit Disk Graph Medium (UDGM – lý tưởng) và UDGM Distance Loss (có suy hao theo khoảng cách). Đề án sử dụng UDGM Distance Loss để mô phỏng thực tế hơn.
- **Scripting:** Hỗ trợ script JavaScript để tự động hóa thực nghiệm và thu thập kết quả.

- **Mote Output:** Ghi log chi tiết từng nút, bao gồm bản tin RPL gửi/nhận, thay đổi parent, giá trị Rank và cảnh báo IDS.

2.5.3 Cấu hình mô phỏng chuẩn

Bảng 2.2 tổng hợp các tham số mô phỏng chung được sử dụng nhất quán trong tất cả các kịch bản kiểm thử của đề án.

Bảng 2.2 Tham số cấu hình mô phỏng Cooja

Tham số	Giá trị
Công cụ mô phỏng	Contiki-NG + Cooja Simulator
Mô hình radio	UDGM: Distance Loss
Số nút	20 nút (1 Border Router + 19 mote)
Phân bố không gian	Ngẫu nhiên trong vùng 200m × 200m
Objective Function	MRHOF (ETX metric)
Tần suất gửi dữ liệu	1 gói / 10 giây
Thời gian mô phỏng	300 giây / kịch bản
Tỉ lệ nút tấn công	0%, 10%, 25%, 50%
Kịch bản đánh giá	Baseline / Under Attack / With IDS

2.6 Tóm tắt chương

Chương này đã trình bày các nền tảng lý thuyết cần thiết cho đề án, bao gồm: (i) nguyên lý hoạt động của RPL với cấu trúc DODAG, bốn loại bản tin điều khiển, cơ chế Trickle Timer và hai Objective Function OF0/MRHOF; (ii) phân loại tấn công RPL theo ba nhóm Resources, Topology và Traffic; (iii) các phương pháp và kiến trúc IDS trong IoT/LLN; (iv) bộ chỉ số đánh giá hiệu năng PDR, E2ED, CO, Power, TPR, FPR; và (v) môi trường mô phỏng Contiki-NG/Cooja.

Các kiến thức này tạo nền tảng để phân tích chi tiết 4 dạng tấn công RPL được trình bày trong Chương 3 và thiết kế IDS ở Chương 4.

3 PHÂN TÍCH CÁC DẠNG TẤN CÔNG RPL

Chương này phân tích chi tiết bốn dạng tấn công RPL được chọn nghiên cứu. Với mỗi dạng tấn công, phần trình bày theo cấu trúc thống nhất: (1) cơ chế hoạt động, (2) tác động đến hiệu năng mạng, (3) dấu hiệu nhận biết và (4) hướng phòng chống. Cuối chương có bảng so sánh tổng hợp bốn kiểu tấn công.

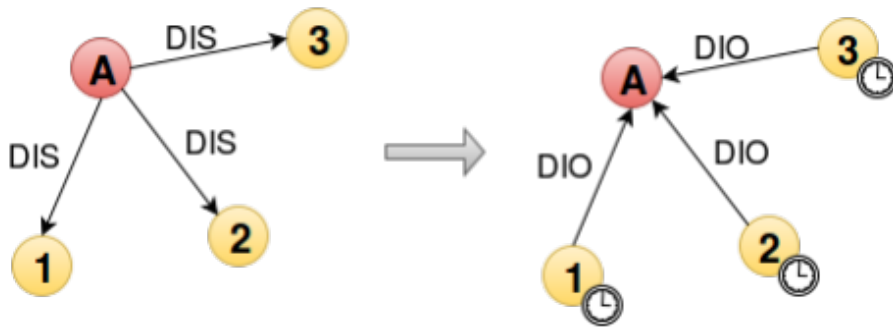
3.1 DIS Flooding Attack

3.1.1 Cơ chế tấn công

DIS Flooding là tấn công thuộc nhóm *Resources – Direct*, khai thác thiết kế của bản tin **DIS** và cơ chế **Trickle Timer** trong RPL [1].

Trong hoạt động bình thường, một nút hợp lệ chỉ gửi DIS khi: (i) không nhận được DIO trong khoảng thời gian quy định (mặc định 30 giây trên Contiki-NG), (ii) nút cha hiện tại bị hỏng, hoặc (iii) phát hiện routing inconsistency. Khi nhận DIS, mọi nút hàng xóm đang là thành viên DODAG phải ngay lập tức **reset Trickle Timer** về I_{min} và phát DIO — thay vì chờ hết chu kỳ hiện tại (có thể lên đến I_{max}).

Kẻ tấn công khai thác quy trình này bằng cách phát DIS liên tục với tần suất rất cao (100 lần/giây, so với 30 giây/lần bình thường). Kết quả là tất cả các nút trong phạm vi thu phát của kẻ tấn công liên tục reset Trickle Timer và phát DIO, gây **bùng nổ bản tin điều khiển** trên toàn mạng. Hình 3.1 minh họa cơ chế này.



Hình 3.1 Cơ chế DIS Flooding Attack: kẻ tấn công A liên tục phát DIS, buộc các nút lân cận reset Trickle Timer và phát DIO

3.1.2 Tác động đến hiệu năng mạng

DIS Flooding gây ra bốn tác động tiêu cực chính:

1. **Tăng Control Overhead:** Số bản tin DIO tăng đột biến. Theo [1], CO tăng từ 500–550 bản tin (baseline) lên hơn 1200 bản tin trong kịch bản 10 kẻ tấn công.

2. **Giảm PDR:** Kênh truyền bị chiếm dụng bởi DIO, gói dữ liệu không được phục vụ kịp thời. PDR giảm từ 97–98% (baseline) xuống 67–71% khi bị tấn công.
3. **Tăng E2E Delay:** Tắc nghẽn kênh và mất ổn định tô-pô kéo dài thời gian truyền. E2ED tăng từ 99–107 ms lên tới 304 ms với 10 kẻ tấn công [1].
4. **Cạn kiệt năng lượng:** Tiêu thụ điện trung bình tăng từ 1.95 mW lên 3.48 mW (tăng 78.5%) khi bị tấn công nặng.

3.1.3 Dấu hiệu nhận biết

- Tần suất nhận DIS từ một nút cụ thể vượt ngưỡng bình thường.
- **DIS Traffic Load (DTL)** — tỉ lệ bản tin DIS trên tổng bản tin nhận được — tăng bất thường:

$$DTL = \frac{N_{DIS}}{N_{total}} \quad (3.5)$$

- Tổng số bản tin DIO tăng đột biến trong phạm vi quan sát.

3.1.4 Phòng chống

Giải pháp **DIS-FDM** [1] sử dụng Dempster-Shafer Theory (DST) kết hợp hai nguồn bằng chứng:

- **Nguồn chính:** Vi phạm liên tiếp trong thống kê DTL so với ngưỡng động T_{DTL} .
- **Nguồn phụ:** Tần suất thay đổi preferred parent bất thường.

Khi tích hợp DIS-FDM: PDR phục hồi lên 87–96%, E2ED giảm về 132–173 ms, tiêu thụ điện giảm xuống 2.28 mW.

3.2 Blackhole Attack

3.2.1 Cơ chế tấn công

Blackhole Attack thuộc nhóm *Topology – Isolation*, là dạng tấn công từ chối dịch vụ (DoS) nguy hiểm trong RPL [5].

Tấn công diễn ra theo hai giai đoạn:

1. **Giai đoạn thu hút (Attraction):** Nút tấn công quảng bá DIO với Rank thấp và/hoặc ETX thấp giả tạo, khiến nhiều nút lân cận chọn nó làm preferred parent. Đây thực chất là kết hợp với Sinkhole Attack hoặc Decreased Rank Attack.

2. **Giai đoạn hủy gói (Dropping):** Sau khi thu hút được lưu lượng, nút tấn công âm thầm loại bỏ **toàn bộ** gói tin cần chuyển tiếp thay vì gửi đến nút gốc. Các nút nạn nhân không nhận được xác nhận, dẫn đến mất gói hàng loạt.

Khi vị trí kẻ tấn công được chọn khéo léo (gần nút gốc), nó có thể cô lập hoàn toàn một nhánh mạng. Một biến thể nguy hiểm hơn là **Selective Forwarding (Greyhole)**: nút độc hại chỉ hủy một tỉ lệ gói (không phải 100%), khó phát hiện hơn.

3.2.2 Tác động đến hiệu năng mạng

1. **Giảm PDR nghiêm trọng:** Các nút kết nối qua kẻ tấn công có PDR giảm về 0–20%. Theo [5], trong mạng 16 nút lossless, hầu hết nút bị ảnh hưởng nhận PDR dưới 20% khi không có IDS.
2. **Cô lập nút:** Các nút chỉ có đường đi qua kẻ tấn công bị cô lập hoàn toàn khỏi nút gốc.
3. **Tăng E2E Delay:** Các nút cố thử lại (retransmit) khi không nhận được ACK, tăng trễ và tiêu tốn năng lượng.
4. **Tăng tiêu thụ điện:** Cơ chế retransmission và route repair tự động của RPL tiêu tốn năng lượng thêm.

3.2.3 Dấu hiệu nhận biết

- Tỉ lệ gửi gói thành công (Packet Delivery Ratio) trên toàn mạng giảm mạnh, trong đó các gói tin dữ liệu từ nút nạn nhân có PDR giảm xuống gần 0.

$$PDR = \frac{N_{\text{send}}}{N_{\text{receive}}} \quad (3.6)$$

3.2.4 Phòng chống

Giải pháp **phân tán dựa trên các bản tin yêu cầu và xác nhận** [5]: Mỗi nút nghe các gói tin đi ra từ nút cha của nó. Sau một khoảng thời gian nhất định, nếu không có gói tin nào, nó sẽ gửi yêu cầu xác nhận đến các nút lân cận. Các nút lân cận sẽ gửi yêu cầu đến nút bị nghi ngờ. Nếu nút nghi ngờ không phản hồi, nó sẽ bị đánh dấu là blackhole và nút con sẽ tìm nút cha khác.

3.3 Decreased Rank Attack

3.3.1 Cơ chế tấn công

Decreased Rank Attack thuộc nhóm *Topology – Sub-optimization*, khai thác cơ chế chọn nút cha dựa trên Rank trong RPL [?].

Nút tấn công quảng bá DIO với giá trị **Rank thấp hơn** Rank thực tế của nó (thậm chí gần bằng Rank của nút gốc). Các nút lân cận, theo Objective Function, ưu tiên chọn nút có Rank thấp nhất làm preferred parent. Hệ quả:

- Nhiều nút cập nhật preferred parent về phía kẻ tấn công.
- Cấu trúc DODAG bị méo: một nhánh cây lớn phụ thuộc vào nút độc hại.
- Kẻ tấn công kiểm soát luồng dữ liệu của các nút nạn nhân, tạo điều kiện thực hiện Blackhole hoặc eavesdropping.
- Với OF0, tấn công đặc biệt hiệu quả vì OF0 không có cơ chế hysteresis — nút luôn chọn parent có Rank thấp nhất.

Một kết quả quan trọng từ [?]: tấn công đặt kẻ tấn công 1–2 hop từ nút gốc có thể **không gây tác động trực tiếp** nhưng làm mất ổn định mạng gián tiếp qua loop, khiến cơ chế IDS dựa trên network profiling khó phát hiện.

3.3.2 Tác động đến hiệu năng mạng

1. **Tạo vòng lặp (Routing Loop):** Cấu trúc DODAG không hợp lệ dẫn đến vòng lặp định tuyến, đặc biệt là khi nút tấn công nhận một nạn nhân của nó là nút cha. Nút nạn nhân phải công bố Rank vô cực để thoát, reset Trickle Timer liên tục.
2. **Giảm PDR:** Khi kẻ tấn công kết hợp với Blackhole hoặc khi loop xảy ra, PDR giảm đáng kể. Theo [?], PDR giảm mạnh hơn khi dùng OF0 so với MRHOF.
3. **Tăng tiêu thụ điện:** Các nút liên kề kẻ tấn công có mức tiêu thụ điện tăng bất thường. Ví dụ: nút 10 tăng lên 4.12 mW (OF0) so với baseline 0.98 mW (Sec-OF).
4. **Mất ổn định tô-pô:** Tần suất thay đổi parent tăng, gây overhead không cần thiết.

3.3.3 Dấu hiệu nhận biết

- Nút quảng bá Rank thấp bất thường so với vị trí vật lý ước tính (số hop từ gốc).
- Tần suất thay đổi preferred parent tăng cao ở các nút lân cận.
- Xuất hiện routing inconsistency (nút con có Rank thấp hơn parent).
- Tiêu thụ điện bất thường tập trung tại các nút quanh kẻ tấn công.

3.3.4 Phòng chống

Secure Objective Function (Sec-OF) [?]: Kết hợp xác thực Rank dựa trên hop count — nút chỉ chấp nhận parent nếu Rank quảng bá nhất quán với khoảng cách hop thực tế. Đồng thời có thể cân nhắc kết hợp chuỗi hash rank trong VeRA [3] để xác thực cả Rank lẫn Version Number. Sec-OF đưa tiêu thụ điện về mức 0.98–1.19 mW, gần với mức baseline.

3.4 Tấn công phiên bản DODAG

3.4.1 Cơ chế tấn công

Tấn công phiên bản DODAG thuộc nhóm *Resources – Indirect*, khai thác cơ chế **DODAGVersionNumber** trong RPL [3].

Trong RPL, chỉ nút gốc có quyền tăng DODAGVersionNumber để khởi tạo tái cấu trúc tô-pô toàn cục (*global repair*). Khi một nút nhận DIO với VersionNumber mới hơn hiện tại, nó: (i) chấp nhận phiên bản mới, (ii) hủy parent set hiện tại, (iii) bắt đầu lại quá trình tham gia DODAG từ đầu, (iv) reset Trickle Timer về I_{min} và phát DIO với version mới.

Kẻ tấn công **giả mạo nút gốc** bằng cách phát DIO với DODAGVersionNumber cao hơn hiện tại. Kết quả: toàn mạng tái cấu trúc đồng thời, tiêu tốn lượng lớn năng lượng và băng thông điều khiển. Nếu kẻ tấn công lặp lại liên tục, mạng không bao giờ ổn định được.

Kết hợp với tấn công này, kẻ tấn công còn có thể quảng bá Rank thấp (Decreased Rank) trong DIO giả, trở thành nút trung gian chiếm lưu lượng eavesdrop toàn bộ mạng.

3.4.2 Tác động đến hiệu năng mạng

1. **Tăng Control Overhead đột biến:** Mỗi lần tái cấu trúc toàn cục phát sinh hàng loạt DIO từ tất cả các nút.
2. **Gián đoạn dịch vụ:** Trong quá trình tái cấu trúc, các nút chưa thiết lập được parent mới không chuyển tiếp dữ liệu, PDR giảm mạnh.
3. **Cạn kiệt pin:** Chi phí năng lượng cho một lần global repair là rất lớn. Tấn công lặp đi lặp lại rút ngắn nghiêm trọng tuổi thọ mạng.
4. **Không ổn định tô-pô kéo dài:** Mạng không đạt được trạng thái ổn định, Trickle Timer liên tục ở mức I_{min} .

3.4.3 Dấu hiệu nhận biết

- DODAGVersionNumber tăng không được khởi tạo từ nút gốc thực sự.

- Nhiều nút đồng loạt hủy parent và bắt đầu tái cấu trúc trong cùng một khoảng thời gian ngắn.
- Số lượng bản tin DIO tăng đột biến toàn mạng (phân biệt với DIS Flooding: ở đây là DIO tăng, không phải DIS).
- Nút nguồn DIO có VersionNumber bất thường không phải địa chỉ của nút gốc thực sự.

3.4.4 Phòng chống

VeRA (Version Number and Rank Authentication) [3] sử dụng hai chuỗi hash để xác thực:

- **Version Number Hash Chain:** Nút gốc tạo chuỗi $V_n, V_{n-1}, \dots, V_1, V_0$ với $V_n = h(r)$, $V_i = h(V_{i+1})$. Mỗi lần tăng VersionNumber, gốc tiết lộ phần tử tiếp theo trong chuỗi. Nút trung gian xác minh bằng cách kiểm tra $h^i(V_i) = V_0$.
- **Rank Hash Chain:** Xác thực tính đơn điệu tăng của Rank theo chuỗi từ gốc xuống lá, ngăn Decreased Rank Attack.

Overhead ước tính trên MICAz: HMAC-SHA-1 tốn 5.6 ms, toàn bộ VNA (Version Number Authentication) tốn 2040 ms ở gốc và 7650 ms ở nút trung gian [3] — chấp nhận được cho LLN.

3.5 So sánh tổng hợp bốn kiểu tấn công

Bảng 3.3 tổng hợp so sánh bốn kiểu tấn công theo các tiêu chí chính, giúp định hướng cho thiết kế IDS ở Chương 4.

Bảng 3.3 So sánh bốn kiểu tấn công RPL được nghiên cứu

Tiêu chí	DIS Flooding	Blackhole	Decreased Rank	VeRA / Version
Nhóm tấn công	Resources	Topology	Topology	Resources
Kiểu	Direct	Isolation	Sub-optim.	Indirect
Bản tin khai thác	DIS	DIO + drop	DIO (Rank)	DIO (Version)
Ảnh hưởng PDR	Trung bình	Nghiêm trọng	Trung bình	Trung bình
Ảnh hưởng năng lượng	Cao	Cao	Cao	Rất cao
Khó phát hiện	Thấp	Trung bình	Cao	Trung bình
Chỉ số phát hiện chính	DTL, DIO rate	PDR, RRR	Rank, parent change	VersionNumber
Giải pháp phòng chống	DIS-FDM	Timer-based	Sec-OF	VeRA

3.6 Tóm tắt chương

Chương này đã phân tích chi tiết bốn dạng tấn công RPL: DIS Flooding Attack, Blackhole Attack, Decreased Rank Attack và VeRA/Version Number Attack. Mỗi kiểu tấn công có cơ chế khai thác khác nhau, tác động đến các chỉ số hiệu năng theo mức độ và chiều hướng riêng biệt. Đặc biệt:

- **DIS Flooding** tăng Control Overhead và giảm PDR thông qua việc bão hòa kênh điều khiển.
- **Blackhole** là tấn công nguy hiểm nhất về mặt mất dữ liệu, có thể đưa PDR của nút nạn nhân về 0.
- **Decreased Rank** gây mất ổn định tô-pô kéo dài và có thể kết hợp với các tấn công khác, đặc biệt khó phát hiện khi đặt kẻ tấn công xa nút gốc.
- **VeRA/Version Attack** gây tổn kém năng lượng nhất vì kích hoạt tái cấu trúc tô-pô toàn cục lặp đi lặp lại.

Hiểu biết về dấu hiệu nhận biết của từng kiểu tấn công sẽ là cơ sở trực tiếp để thiết kế các module phát hiện trong IDS được trình bày ở Chương 4.

4 THIẾT KẾ VÀ TRIỂN KHAI IDS PHÁT HIỆN TẤN CÔNG RPL

Chương này trình bày thiết kế và triển khai hệ thống IDS phát hiện tấn công RPL. Hệ thống được xây dựng dựa trên việc phân tích gói tin và so sánh với quy tắc nhằm phát hiện bất thường trong giao thức RPL. Hệ thống được triển khai trực tiếp trong mã chương trình nạp vào thiết bị, hướng đến sự gọn nhẹ, ít ảnh hưởng đến tốc độ xử lý.

4.1 Phòng chống tấn công DIS flooding

Như đã giới thiệu ở phần 3.1, đồ án xây dựng hệ thống IDS dựa trên giải pháp DIS-FDM đối với dạng tấn công làm ngập DIS. Tác giả chọn ngưỡng tối đa cho phép xử lý gói tin DIS là $DTL = \frac{1}{2}$. Khi mới khởi động nút, DTL cũng được thiết lập để mang giá trị trên. Mỗi khi một gói tin được chấp nhận để xử lý, mẫu số sẽ được cộng thêm 1. Tiếp đến, nếu gói tin được chấp nhận là bản tin DIS, tử số cũng sẽ được cộng thêm 1. Nếu tỷ lệ DTL vượt quá ngưỡng 0.5, nút sẽ từ chối xử lý các gói tin DIS tiếp theo cho đến khi DTL giảm xuống dưới ngưỡng. Sau khoảng thời gian 5 giây, tổng số gói tin và số gói tin DIS sẽ được đặt lại về 0 (DTL quay về lại giá trị $\frac{1}{2}$), cho phép nút tiếp tục xử lý các gói tin DIS mới.

Thuật toán phòng chống được triển khai dưới dạng mã nguồn C với bộ thư viện contiki-ng, được chia làm hai phần chính: một phần xử lý gói tin và một phần định kỳ đặt lại giá trị DTL.

Phần xử lý gói tin (đoạn mã 4.1) kiểm tra loại gói tin, kiểm tra DTL để đưa ra quyết định với gói tin DIS, và cập nhật DTL dựa trên số lượng gói tin đã xử lý. Phần này được triển khai dưới dạng một hàm được gọi trước khi gói tin đi vào quá trình xử lý chính thức của hệ thống mạng, sử dụng thư viện netstack của contiki-ng để can thiệp vào quá trình xử lý gói tin.

```
static enum netstack_ip_action
ip_input(void)
{
    uint8_t proto = 0;
    uipbuf_get_last_header(uip_buf, uip_len, &proto);
    if (proto != UIP_PROTO_ICMP6 ||
        UIP_ICMP_BUF->type != ICMP6_RPL ||
        UIP_ICMP_BUF->icode != RPL_CODE_DIS) {
        total_packets++;
    }
}
```

```

        return NETSTACK_IP_PROCESS;
    }
    float dis_ratio = (float)(dis_packets + 1)
                      / (float)(total_packets + 2);
    if (dis_ratio > RPL_DIS_PREVENTION_THRESHOLD) {
        LOG_INFO("Dropping DIS packet. From: ");
        LOG_INFO_6ADDR(&UIP_IP_BUF->srcipaddr);
        LOG_INFO_(" DIS ratio: %u%%\n",
                  (unsigned int)(dis_ratio * 100));
        return NETSTACK_IP_DROP;
    }
    total_packets++;
    dis_packets++;
    LOG_INFO("Accepting DIS packet. From: ");
    LOG_INFO_6ADDR(&UIP_IP_BUF->srcipaddr);
    LOG_INFO_(" DIS ratio: %u%%\n", (unsigned int)(dis_ratio * 100));
    return NETSTACK_IP_PROCESS;
}
[...]
struct netstack_ip_packet_processor packet_processor = {
    .process_input = ip_input,
    .process_output = ip_output
};
[...]
// tiến trình chính
PROCESS_THREAD(udp_client_process, ev, data)
{
    [...]
    PROCESS_BEGIN();
    // Đăng ký bộ xử lý gói tin tùy chỉnh
    netstack_register_ip_packet_processor(&packet_processor);
    [...]
    PROCESS_END();
}

```

Phần đặt lại giá trị DTL (đoạn mã 4.1) sẽ được thực hiện định kỳ sau mỗi 5 giây để đảm bảo hệ thống có thể tiếp tục hoạt động bình thường sau khi đã từ chối một số lượng gói tin DIS nhất định. Phần này được triển khai dưới dạng một tiến trình riêng biệt khỏi tiến trình chính của node. `etimer_set(&periodic_timer, CLOCK_SECOND`

* 5); được sử dụng để thiết lập bộ đếm thời gian, và
PROCESS_WAIT_EVENT_UNTIL(etimer_expired(&periodic_timer)); sẽ chờ cho đến
khi bộ đếm thời gian hết hạn trước khi thực hiện việc đặt lại giá trị DTL.

```
PROCESS(dis_counter, "DIS Periodic Counter");
```

```
PROCESS_THREAD(dis_counter, ev, data)
```

```
{
```

```
    static struct etimer periodic_timer;
```

```
    PROCESS_BEGIN();
```

```
    while (1)
```

```
    {
```

```
        etimer_set(&periodic_timer, CLOCK_SECOND * 5);
```

```
        PROCESS_WAIT_EVENT_UNTIL(etimer_expired(&periodic_timer));
```

```
        total_packets = 0;
```

```
        dis_packets = 0;
```

```
        etimer_reset(&periodic_timer);
```

```
    }
```

```
    PROCESS_END();
```

```
}
```

4.2 Phòng chống tấn công phiên bản DAG

4.3 Thiết kế

Đồ án xây dựng cơ chế phòng chống tấn công phiên bản DAG dựa trên giải pháp VeRa. Tuy nhiên, do hạn chế về thời gian và năng lực, tập trung vào việc phát hiện và phản ứng với các gói tin RPL có phiên bản DAG không hợp lệ và bỏ qua phần kiểm tra rank được nhắc đến trong bài đề xuất VeRa.

Quá trình hoạt động của giải pháp được triển khai trong đồ án bao gồm các bước sau:

1. Khi nút gốc bắt đầu hoạt động, nó khởi tạo một chuỗi hash lưu trữ các phiên bản DAG hợp lệ.
2. Nút gốc lấy phiên bản DAG đầu tiên từ giá trị cuối cùng của chuỗi hash.
3. Việc gửi DIO sẽ được thực hiện như trong một mạng DIO bình thường.

4. Trong quá trình khởi tạo mạng (các nút còn lại chưa lưu trữ phiên bản DAG), nút sẽ lấy phiên bản DAG từ gói tin DIO đầu tiên mà nó nhận được và lưu trữ nó như một phiên bản DAG hợp lệ.
5. Sau khi tham gia mạng, cơ chế kiểm tra phiên bản bắt đầu hoạt động.
6. Khi nhận gói DIO chứa phiên bản DAG khác với phiên bản DAG hiện tại được lưu trong nút, nút sẽ thực hiện hàm hash với số phiên bản DAG mới và so sánh với phiên bản DAG hiện tại. Nếu kết quả hash không khớp, nút sẽ từ chối gói DIO và đưa nút gửi vào danh sách đen.
7. Nếu nút gửi bị đưa vào danh sách đen, nút sẽ từ chối tất cả các gói tin RPL từ nút đó.

Quá trình trên giả định rằng nút gốc không bị giả, cũng như quá trình tấn công chỉ xảy ra sau khi mạng đã được khởi tạo và các nút đã lưu trữ phiên bản DAG hợp lệ. Nhóm tác giả không xét đến trường hợp tấn công trong giai đoạn khởi tạo mạng.

4.3.1 Triển khai

Giải pháp được triển khai dưới dạng mã nguồn C với bộ thư viện `contiki-ng`, tương tự như phần phòng chống tấn công DIS flooding. Mã triển khai bao gồm phần xử lý gói tin tại `netstack` để kiểm tra phiên bản DAG trong gói DIO, và một bản tùy chỉnh của thư viện `rpl-lite` để quản lý chuỗi hash.

Tác giả đã thêm mảng chứa các phiên bản DAG hợp lệ, gồm 32 phần tử:

Trong file `rpl-lite/rpl-const.h`:

```
[...]  
#define MAX_DAG_VER_HASH_ENTRIES 32  
[...]
```

Tại nút gốc, các phiên bản DAG hợp lệ sẽ được khởi tạo và lưu trữ trong một mảng. Một biến đếm sẽ được sử dụng để theo dõi chỉ số hiện tại trong mảng.

```
[...]  
// Mảng lưu trữ các phiên bản DAG hợp lệ  
// cùng biến đếm chỉ số hiện tại trong mảng  
static uint8_t dag_versions[MAX_DAG_VER_HASH_ENTRIES];  
static uint8_t dag_ver_index;  
[...]
```


Một hàm hash đơn giản được sử dụng tại cả nút gốc và các nút khác. Tại nút gốc, hàm hash được dùng để khởi tạo phần tử tiếp theo trong chuỗi hash. Tại nút con, hàm hash sẽ dùng để kiểm tra xem phiên bản mới có hash ra phiên bản hiện tại hay không. Do phiên bản DAG chỉ có 8 bit, hàm hash được thiết kế để đảm bảo số phiên bản luôn nằm trong khoảng 0-255. Hàm hash được triển khai như sau:

```
[...]
uint8_t dag_ver_hash(uint8_t input)
{
    return (input * 31) % 256;
}
[...]
```

Chuỗi hash được khởi tạo tại nút gốc khi bắt đầu hoạt động:

```
[...]
void dag_ver_chain_init(uint8_t input_value) {
    dag_versions[MAX_DAG_VER_HASH_ENTRIES - 1]
        = dag_ver_hash(input_value);
    for (int i = MAX_DAG_VER_HASH_ENTRIES - 2; i >= 0; i--)
    {
        dag_versions[i] = dag_ver_hash(dag_versions[i + 1]);
    }
}
[...]
```

Khi nút gốc cần thực hiện global repair, nó sẽ cập nhật phiên bản DAG mới bằng cách lấy giá trị tiếp theo trong chuỗi hash:

```
dag_ver_index = (dag_ver_index + 1) % MAX_DAG_VER_HASH_ENTRIES;
curr_instance.dag.version = dag_versions[dag_ver_index];
```

Điều kiện phát hiện phiên bản không hợp lệ đối với các nút con cũng thay đổi từ phiên bản trên DIO có số nhỏ hơn phiên bản DAG hiện tại sang việc so sánh kết quả hash của phiên bản mới với phiên bản hiện tại:

```
// lấy giá trị hash phiên bản mới
// khoảng cách tối đa là 10 lần hash
uint8_t newver = dio->version;
```

```

for (int i = 0; i < 10; i++) {
    if (newver == curr_instance.dag.version) {
        break;
    }
    newver = dag_ver_hash(newver);
}
// nếu sau 10 lần hash vẫn không khớp,
// coi như phiên bản mới không hợp lệ
if(newver != curr_instance.dag.version) {
    LOG_INFO("Invalid DAG version from ");
    LOG_INFO_6ADDR(from);
    LOG_INFO_(", our version %u, recv %u\n",
        curr_instance.dag.version, dio->version);
    // Nếu đó là nút gốc gửi gói DIO với phiên bản không hợp lệ,
    // giả định đó là gói tin hợp lệ và thực hiện reset phiên bản.
    if(dio->rank == ROOT_RANK) {
        rpl_timers_dio_reset("Heard old version from root");
    }
    return;
}

```

Trong phạm vi đề án, nhóm tác giả chưa hoàn thiện việc tạo chuỗi hash mới khi chỉ số đếm đạt đến giới hạn của mảng. Tuy nhiên, quá trình mô phỏng về cơ bản sẽ không kéo dài tới thời điểm này, do đó khía cạnh này sẽ tạm thời được bỏ qua.

Đối với phần phát hiện gói tin, nhóm tác giả sử dụng thư viện netstack để can thiệp vào quá trình xử lý gói tin RPL.

```

[...]
if (uip_buf[UIP_IPH_LEN + uip_ext_len + 5]
    != curr_instance.dag.version)
{
    if (dag_ver_hash(uip_buf[UIP_IPH_LEN + uip_ext_len + 5])
        != curr_instance.dag.version)
    {
        if (!rpl_dag_root_is_root())
        {
            if (!curr_instance.dag.preferred_parent) {
                return NETSTACK_IP_PROCESS;
            }
        }
    }
}

```

```

    }

    if (!uip_ip6addr_cmp(&UIP_IP_BUF->srcipaddr,
                        &root_ipaddr)
        && uip_ip6addr_cmp(&UIP_IP_BUF->srcipaddr,
                        rpl_neighbor_get_ipaddr
                            (curr_instance.dag.preferred_parent)
                        )
    ) {
        // xoá preferred parent
        nbr_table_remove(rpl_neighbors,
                        curr_instance.dag.preferred_parent);
        curr_instance.dag.preferred_parent = NULL;
        rpl_dag_update_state();
        need_version_repair = 1;
    }
    if (!uip_ip6addr_cmp(&UIP_IP_BUF->srcipaddr,
                        &blacklisted_ipaddr))
    {
        // thêm vào blacklist
        uip_ip6addr_copy(&blacklisted_ipaddr,
                        &UIP_IP_BUF->srcipaddr);
        blacklist_timer_needs_reset = true;
    }
}
return NETSTACK_IP_DROP;
}
[...]
```

4.4 Phòng chống tấn công blackhole

4.5 Thiết kế

Quá trình phòng chống tấn công dựa trên cơ chế tin tưởng - nghi ngờ - xác nhận. Quá trình này được áp dụng cho các nút con sau khi tìm được preferred parent, gồm các bước sau:

1. Sau khi xác định preferred parent, nút con sẽ tạm thời tin cậy và theo dõi các gói tin không phải RPL có địa chỉ nguồn lớp liên kết là preferred parent trong khoảng thời

gian 5 giây. Nếu nhận được gói tin, thời gian chờ sẽ được đặt lại.

2. Nếu sau khoảng thời gian chỉ định trước (trong đề án này là 5 giây) không nhận được gói tin nào, nút con sẽ đánh dấu preferred parent là nghi ngờ và quảng bá gói tin UDP chứa bản tin yêu cầu xác thực (verification-request) đến các nút lân cận, đồng thời đặt một khoảng thời gian chờ.
3. Các nút lân cận khi nhận yêu cầu sẽ đồng loạt gửi bản tin yêu cầu xác thực đến nút bị nghi ngờ (suspect-verification-request).
4. Nút bị nghi ngờ khi nhận được yêu cầu xác thực (suspect-verification-request) sẽ phản hồi bằng một gói tin chứa bản tin xác nhận rằng nó không phải là nút blackhole (suspect-reply).
5. Nút con nhận được bản tin suspect-reply sẽ gửi bản tin xác nhận (verification-confirmation) đến các nút đã gửi yêu cầu đầu tiên (verification-request) để thông báo rằng preferred parent không phải là blackhole.
6. Nếu sau một khoảng thời gian được chỉ định (trong trường hợp này là 10 giây) kể từ lúc gửi verification-request, nút con không nhận được bất kỳ bản tin xác nhận nào, nó sẽ đánh dấu preferred parent là blackhole và từ chối tất cả các gói tin từ nút đó, đồng thời gỡ nút bị đánh dấu khỏi vị trí preferred parent cũn như khỏi bảng các nút lân cận.
7. Nút con sau đó sẽ tìm kiếm preferred parent mới, và lặp lại quá trình trên.

4.5.1 Triển khai

Tác giả xây dựng cấu trúc các gói tin xác thực như sau:

```
// bản tin verification-request sẽ được
// gửi đến các nút lân cận để yêu cầu xác thực preferred parent
struct verification_request_packet {
    uint8_t type;
    // địa chỉ ip của nút bị nghi ngờ
    uip_ip6addr_t suspect_ip;
};
typedef struct verification_request_packet
    verification_req_t;

// bản tin suspect-verification-request sẽ được các nút
// nhận verification-request gửi đến nút bị nghi ngờ để
```

```

// yêu cầu nó chứng minh rằng nó không phải là blackhole
struct suspect_verification_request_packet {
    uint8_t type;
    // địa chỉ ip của nút gửi verification-request
    uip_ip6addr_t initiator_ip;
};
typedef struct suspect_verification_request_packet
    suspect_verification_req_t;

// bản tin suspect-reply sẽ được nút nhận suspect-verification-request
// gửi đến các nút đã gửi yêu cầu để xác nhận rằng nó không phải
// là blackhole
struct suspect_clean_response_packet {
    uint8_t type;
    // địa chỉ ip của nút gửi verification-request
    uip_ip6addr_t initiator_ip;
    // do nút bị nghi ngờ chính là nút gửi bản tin này,
    // địa chỉ của nút bị nghi ngờ sẽ là địa chỉ nguồn
    // trong header ipv6, nên không cần lưu trong bản tin này.
};
typedef struct suspect_clean_response_packet
    suspect_clean_resp_t;

// nút nhận suspect-reply sẽ gửi bản tin xác nhận đến nút
// đã gửi verification-request để thông báo rằng preferred parent
// không phải là blackhole
struct verification_confirmation_packet {
    uint8_t type;
    // địa chỉ ip của nút bị nghi ngờ
    uip_ip6addr_t suspect_ip;
};
typedef struct verification_confirmation_packet
    verification_conf_t;

```

Chương trình phòng chống tấn công blackhole sẽ được triển khai thành 3 phần: phần netstack, tiến trình chuyển trạng thái giữa tin tưởng và nghi ngờ, và tiến trình chuyển trạng thái giữa nghi ngờ và xác nhận nút hợp lệ/blackhole.

Phần netstack input sẽ đọc các gói tin không phải RPL và reset thời gian tin tưởng

nếu nhận được gói tin từ preferred parent:

```
static enum netstack_ip_action
ip_input(void)
{
    uint8_t proto = 0;
    uipbuf_get_last_header(uip_buf, uip_len, &proto);

    // huỷ các gói tin đến từ nút bị đánh dấu là blackhole
    if (uip_ip6addr_cmp(&UIP_IP_BUF->srcipaddr,
        &blacklisted_ipaddr))
    {
        return NETSTACK_IP_DROP;
    }

    // cho các gói tin RPL đi qua,
    // không can thiệp vào quá trình xử lý
    if (proto == UIP_PROTO_ICMP6)
    {
        return NETSTACK_IP_PROCESS;
    }

    if (curr_instance.dag.preferred_parent &&
        linkaddr_cmp(packetbuf_addr(PACKETBUF_ADDR_SENDER),
            rpl_neighbor_get_lladdr(curr_instance.dag.preferred_parent))
    )
    {
        // nếu nhận được gói tin
        // không phải RPL từ preferred parent,
        // đánh dấu flag để chuẩn bị
        // reset bộ đếm ngược thời gian tin tưởng
        trust_timer_reset = true;
    }
    return NETSTACK_IP_PROCESS;
}
```

Phần tiến trình chuyển trạng thái giữa tin tưởng và nghi ngờ sẽ theo dõi thời gian tin tưởng và gửi yêu cầu xác thực nếu thời gian này hết hạn:

```

// đợi cho đến khi có preferred parent
while(!(NETSTACK_ROUTING.node_is_reachable()
    && curr_instance.dag.preferred_parent))
{
    PROCESS_WAIT_EVENT_UNTIL(etimer_expired(&startup_timer));
    etimer_reset(&startup_timer);
}

// khi đã tìm thấy preferred parent,
// bắt đầu tin tưởng và theo dõi
suspected_node_is_safe = true;
etimer_set(&trust_timer, TRUST_SECONDS * CLOCK_SECOND);

while (1)
{
    // đợi đến khi preferred parent được báo là an toàn
    // để bắt đầu theo dõi
    PROCESS_WAIT_EVENT_UNTIL(suspected_node_is_safe);

    // đợi đến khi có yêu cầu reset thời gian tin tưởng
    // hoặc thời gian tin tưởng hết hạn
    PROCESS_WAIT_EVENT_UNTIL(trust_timer_reset ||
        etimer_expired(&trust_timer));

    // nếu cờ trust_timer_reset được đặt,
    // reset bộ đếm thời gian tin tưởng
    // và tiếp tục theo dõi ở vòng lặp tiếp theo
    if (trust_timer_reset)
    {
        etimer_reset(&trust_timer);
        trust_timer_reset = false;
        continue;
    }
    // nếu thời gian tin tưởng hết hạn,
    // dựng bản tin verification-request
    // và multicast đến các nút lân cận
    else if (etimer_expired(&trust_timer))
    {

```

```

verification_req_t verif_req;
verif_req.type = VERIFICATION_REQ_TYPE;
verif_req.suspect_ip = *rpl_neighbor_get_ipaddr(curr_instance.dag.preferred
simple_udp_sendto(&verif_udp_conn, &verif_req, sizeof(verif_req), &rpl_mu
// tắt cờ tin tưởng, chuyển sang trạng thái nghi ngờ
suspected_node_is_safe = false;
continue;
}
}

```

Phần tiến trình chuyển trạng thái giữa nghi ngờ và xác nhận nút hợp lệ/blackhole sẽ theo dõi thời gian chờ xác nhận và cập nhật trạng thái của preferred parent dựa trên việc có nhận được xác nhận hay không:

```

while(1)
{
    // đợi cho đến khi có preferred parent
    while (finding_new_parent)
    {
        // khi đã tìm thấy preferred parent,
        // phá vỡ vòng lặp,
        // bắt đầu tin tưởng và theo dõi
        if (curr_instance.dag.preferred_parent) {
            finding_new_parent = false;
            suspected_node_is_safe = true;
            trust_timer_reset = true;
            break;
        }
        PROCESS_WAIT_EVENT_UNTIL(etimer_expired(&startup_timer));
        etimer_reset(&startup_timer);
    }

    // đợi đến khi thời gian chờ xác nhận hết hạn
    // hoặc cờ suspected_node_is_safe được đặt,
    // xác nhận preferred parent là an toàn
    PROCESS_WAIT_EVENT_UNTIL(etimer_expired(&suspect_timer)
        || suspected_node_is_safe);

    // nếu preferred parent là nút gốc, bỏ qua việc đánh giá

```



```

// giả định rằng nút gốc không bị giả mạo
if (curr_instance.dag.preferred_parent
    && curr_instance.dag.preferred_parent->rank == ROOT_RANK)
{
    continue;
}
// nếu cờ suspected_node_is_safe được đặt,
// reset bộ đếm thời gian nghi ngờ
if (suspected_node_is_safe)
{
    etimer_reset(&suspect_timer);
}
// ngược lại, nếu thời gian chờ xác nhận
// hết hạn mà không nhận được bản tin
// verification-confirmation
else if (etimer_expired(&suspect_timer))
{
    // lưu lại địa chỉ IP của nút bị nghi ngờ vào blacklist
    uip_ip6addr_copy(&blacklisted_ipaddr,
        rpl_neighbor_get_ipaddr
            (curr_instance.dag.preferred_parent)
    );

    // bỏ preferred parent khỏi bảng các nút lân cận
    nbr_table_remove(rpl_neighbors,
        curr_instance.dag.preferred_parent);

    // gỡ trạng thái preferred parent
    curr_instance.dag.preferred_parent = NULL;

    // cập nhật lại trạng thái DAG để tìm preferred parent mới
    rpl_dag_update_state();

    // đánh dấu cần tìm preferred parent mới
    finding_new_parent = true;

    // đặt lại bộ đếm thời gian chờ xác nhận
    etimer_set(&startup_timer, CLOCK_SECOND);
}

```

}

}

4.6 Phòng chống tấn công hạ Rank (giả mạo rank thấp hơn)

4.6.1 Thiết kế

Thay vì sử dụng phương pháp hash được đề xuất trong bài VeRa, đồ án triển khai một cơ chế đơn giản hơn để phát hiện tấn công hạ Rank bằng cách thêm điều kiện so sánh số bước nhảy. Việc kiểm tra rank sẽ chia thành hai giai đoạn chính: giai đoạn khởi tạo mạng và giai đoạn ổn định sau khi mạng đã được khởi tạo.

Tại giai đoạn khởi tạo mạng (đồ án sẽ đặt khoảng thời gian 2 phút đầu), hệ thống được giả định là không xảy ra tấn công. Do đó, nút sẽ chấp nhận rank từ các gói DIO mà nó nhận được và lưu trữ rank của nút gửi làm rank hợp lệ. Khi nhận gói DIO mới, nút sẽ kiểm tra điều kiện chi phí đường đi dựa trên rank và chỉ số đường truyền ETX. Nếu chi phí đường đi đến gói vừa gửi DIO thấp hơn chi phí đường đi đến preferred parent hiện tại một khoảng lớn hơn ngưỡng cho trước (đã được định nghĩa sẵn trong thư viện của contiki-ng), nút sẽ chấp nhận gói DIO đó và cập nhật rank hợp lệ mới. Nếu không, nút sẽ từ chối gói DIO đó.

Khi đã xác định được preferred parent và preferred parent không đổi trong 30 giây, cũng như đã qua thời gian khởi tạo mạng, nút sẽ chuyển sang giai đoạn ổn định. Lúc này điều kiện sẽ được siết chặt hơn. Ngoài điều kiện chi phí đường đi, nút sẽ chỉ chấp nhận cập nhật preferred parent nếu số bước nhảy từ gói gửi DIO đến nút gốc thấp hơn số bước nhảy từ preferred parent hiện tại đến nút gốc.

4.6.2 Triển khai

Đối với giai đoạn khởi tạo mạng, do thư viện rpl-lite đã có sẵn cơ chế kiểm tra chi phí đường đi, nhóm tác giả không can thiệp gì thêm vào mã nguồn. Đối với giai đoạn ổn định, nhóm tác giả đã thêm điều kiện so sánh số bước nhảy vào dưới dạng một option mới trong gói DIO. Option này sẽ được thêm vào gói DIO tại nút gửi, và được kiểm tra tại nút nhận.

Trong file `rpl-lite/rpl-types.h`, nhóm tác giả thêm `hops_count` vào cấu trúc DAG và neighbor:

```
struct rpl_dag {  
    [...]  
    // số bước nhảy từ DAG root đến node này  
    uint8_t hops_count;  
};
```

```

struct rpl_nbr {
    [...]
    // số bước nhảy từ DAG root đến neighbor này
    uint8_t hops_count;
};

```

Trong file `rpl-lite/rpl-icmp6.h`, nhóm tác giả thêm option mới vào gói DIO tại nút gửi:

```

struct rpl_dio {
    [...]
    // option mới chứa số bước nhảy từ node gửi DIO đến DAG root
    uint8_t hops_count;
};

```

Quá trình lấy giá trị `hops_count` từ gói tin được bổ sung trong `rpl-lite/rpl-icmp6.c` tại hàm xử lý gói DIO:

```

static rpl_nbr_t *
update_nbr_from_dio(uiplib_addr_t *from, rpl_dio_t *dio)
{
    [...]
    switch(subopt_type) {
        [...]
        case RPL_OPTION_HOPS:
            dio.hops_count = buffer[i + 2];
            break;
        [...]
    }
    [...]
}

```

Cũng trong file `rpl-lite/rpl-icmp6.c`, `hops_count` được tính toán và thêm vào gói DIO tại hàm gửi DIO:

```

void
rpl_icmp6_dio_output(uiplib_addr_t *uc_addr)
{
    [...]
}

```

```

#ifdef RPL_WITH_HOP_COUNT

buffer[pos++] = RPL_OPTION_HOPS;
buffer[pos++] = 1;
buffer[pos++] = curr_instance.dag.hops_count;

#endif /* RPL_WITH_HOP_COUNT */
[...]
```

Hàm kiểm tra hops_count hợp lệ được thêm vào trong file rpl-lite/rpl-mrhof.c:

```

static int valid_hop_count(rpl_nbr_t *nbr) {
// nếu neighbor không tồn tại, coi như không hợp lệ
if (nbr == NULL) {
    return 0;
}
// nếu neighbor có hops_count là 0 nhưng rank lại là ROOT_RANK,
// coi như hợp lệ (trường hợp neighbor là nút gốc)
if (nbr->hops_count == 0 && nbr->rank == ROOT_RANK) {
    return 1;
}
// nếu neighbor không phải nút gốc,
// chỉ coi như hợp lệ nếu hops_count lớn hơn 0
else {
    return nbr->hops_count > 0;
}
// mặc định coi như không hợp lệ
return 0;
}
```

Trong hàm best_parent, sau khi loại bỏ các neighbor không hợp lệ về chi phí đường đi, nhóm tác giả thêm điều kiện so sánh hops_count để chọn parent mới:

```

static rpl_nbr_t *
best_parent(rpl_nbr_t *nbr1, rpl_nbr_t *nbr2)
{
    [...]
    if (stabilized_parent) {
```

```

        if (nbr1 == curr_instance.dag.preferred_parent &&
            (!valid_hop_count(nbr2)
             || (nbr1->hops_count <= nbr2->hops_count)))
        {
            return nbr1;
        }
        if (nbr2 == curr_instance.dag.preferred_parent &&
            (!valid_hop_count(nbr1)
             || (nbr2->hops_count <= nbr1->hops_count)))
        {
            return nbr2;
        }
    }

    // từ đây trở xuống là phần mã nguồn gốc
    LOG_INFO("Comparing two acceptable
              parents with path costs %u and %u\n",
              nbr_path_cost(nbr1), nbr_path_cost(nbr2));
    return nbr_path_cost(nbr1) < nbr_path_cost(nbr2)
        ? nbr1 : nbr2;
}

```

Một tiến trình cũng được tạo ra để theo dõi sự ổn định của preferred parent, dựa trên việc preferred parent có thay đổi hay không trong khoảng thời gian 30 giây. Nếu preferred parent thay đổi, tiến trình sẽ đặt lại bộ đếm thời gian. Nếu preferred parent không thay đổi trong 30 giây, tiến trình sẽ đặt cờ để chuyển sang giai đoạn ổn định.

```

while (1)
{
    etimer_set(&periodic_timer, CLOCK_SECOND * 1);
    PROCESS_WAIT_EVENT_UNTIL(etimer_expired(&periodic_timer));

    // cập nhật thời gian ổn định của preferred parent
    // cập nhật theo từng giây
    better_parent_interval = curr_instance.dag.preferred_parent
        ? (clock_time() -
           curr_instance.dag.preferred_parent->better_parent_since)
        : 0;
}

```

```

// nếu preferred parent không thay đổi trong 30 giây,
// coi như đã ổn định, đặt cờ để siết chặt điều kiện chọn parent
if (curr_instance.dag.preferred_parent &&
    better_parent_interval > 30 * CLOCK_SECOND) {
    stabilized_parent = true;
}
}

```

4.7 Tổng kết

Trong chương này, nhóm tác giả đã trình bày chi tiết về thiết kế và triển khai các hệ thống IDS để phát hiện từng loại tấn công phổ biến trong giao thức RPL, bao gồm tấn công làm ngập DIS, tấn công phiên bản DAG, tấn công blackhole và tấn công hạ Rank. Mỗi giải pháp được xây dựng dựa trên các cơ chế khác nhau, bao gồm: theo dõi tỷ lệ gói tin, sử dụng chuỗi băm, sử dụng cơ chế tin tưởng - nghi ngờ - xác nhận, hoặc với dạng tấn công giảm rank, chỉ đơn giản là thêm một điều kiện kiểm tra gói tin DIO. Các giải pháp này được triển khai trực tiếp trong mã nguồn của từng nút tham gia.

5 THỰC NGHIỆM VÀ ĐÁNH GIÁ

Trong chương này, nhóm sẽ trình bày về quá trình thực nghiệm và đánh giá kết quả của đề tài. Các bước thực nghiệm sẽ được mô tả chi tiết, bao gồm việc thiết lập môi trường thử nghiệm, thu thập dữ liệu, và phân tích kết quả. Ngoài ra, nhóm cũng sẽ so sánh kết quả đạt được với các phương pháp hiện có để đánh giá hiệu quả của giải pháp đề xuất.

5.1 Môi trường thực nghiệm

Phần cứng:

- Máy vi tính: Máy ảo trên nền tảng VirtualBox 7.2.4
- CPU: 4 nhân
- RAM: 8 GB
- Kích thước swap: 4 GB

Phần mềm:

- Hệ điều hành: Xubuntu 24.04 LTS
- Các công cụ và thư viện cơ bản: git, git-lfs, GCC, doxygen, curl, wireshark, python3-serial, srecord, rlwrap, openjdk-17-jdk (để sử dụng cooja)
- Framework cho firmware: Contiki-NG bản develop

5.2 Các tiêu chí đánh giá

Để đánh giá hiệu quả của giải pháp đề xuất, nhóm sẽ sử dụng các tiêu chí sau:

- Độ chính xác trong việc phát hiện tấn công: Dựa trên tỉ lệ giữa phát hiện đúng và tổng số lần phát hiện.
- Tỉ lệ truyền dữ liệu thành công: Dựa trên tỉ lệ gói tin đến đích so với tổng số gói tin gửi đi từ tất cả các nút. Khi bị tấn công, tỉ lệ này có thể giảm đi. Việc phòng chống giúp hạn chế sự hao hụt này.
- Chi phí gói tin điều khiển RPL: Dựa trên số lượng gói tin RPL được gửi đi trong quá trình hoạt động của mạng. Một số dạng tấn công có thể làm tăng số lượng gói tin điều khiển (chẳng hạn, tấn công phiên bản, tấn công DIS, và tấn công giảm Rank). Việc phòng chống tấn công được kỳ vọng sẽ làm giảm độ gia tăng này.

- **Mức tiêu thụ năng lượng:** Một số thiết bị nhúng có thể có hai chế độ hoạt động: chế độ tiết kiệm năng lượng (Low-Power Mode) và chế độ hoạt động thông thường. Khi bị tấn công, tần suất và thời gian thiết bị phải ở trong trạng thái hoạt động tăng lên. Ngoài ra, thời gian hoạt động của bộ thu phát cũng tăng lên. Cả hai điều trên đều đồng nghĩa với tiêu thụ năng lượng nhiều hơn. Việc phòng chống tấn công được kỳ vọng sẽ đưa mức tiêu thụ năng lượng trở lại gần nhất có thể với mức tiêu thụ của mạng không bị tấn công.

5.3 Kịch bản thực nghiệm

Nhóm thực hiện 10 kịch bản thực hiện, bao gồm kịch bản hoạt động bình thường, 4 kịch bản tấn công (DIS flooding, tấn công phiên bản, tấn công giảm Rank, và tấn công blackhole) cùng các kịch bản phòng chống tương ứng. Riêng với nhóm kịch bản tấn công blackhole, do khác biệt về đồ thị mạng, nhóm cần triển khai thêm một kịch bản hoạt động bình thường đối với kịch bản này.

Với các kịch bản tấn công DIS flooding, tấn công phiên bản, và tấn công giảm Rank, nhóm sử dụng mô hình mạng như hình, với nút tấn công (ID 8) nằm ở cuối đồ thị.

Riêng với kịch bản tấn công blackhole, nhóm sử dụng mô hình mạng như hình, với nút tấn công (ID 8) nằm ở giữa đồ thị. Cách bố trí khác biệt này là do việc đặt nút tấn công ở cuối đồ thị đồng nghĩa với việc không có gói tin nào đi qua nó, dẫn đến việc không có tấn công nào xảy ra để đánh giá.

Trong tất cả các kịch bản, nút tấn công sẽ được cấu hình để "ngủ" trong 2 phút đầu tiên, sau đó sẽ thực hiện tấn công trong 28 phút tiếp theo. Điều này dựa trên giả định rằng khả năng xảy ra tấn công trong 2 phút đầu tiên là rất thấp, do đó nhóm sẽ bỏ qua dữ liệu thu thập được trong khoảng thời gian này để tránh ảnh hưởng đến kết quả đánh giá.

5.4 Kết quả

Bảng 5.4 tổng hợp kết quả về độ chính xác trong việc phát hiện tấn công. Việc phát hiện nút và lưu lượng tấn công có tỉ lệ từ 95% đến 100% là do việc đánh giá độ chính xác dựa trên các đặc điểm nhận diện rõ ràng của các loại tấn công. Tuy nhiên, trong môi trường thực tế với nhiều yếu tố nhiễu, tỉ lệ này có thể thấp hơn.

Bảng 5.5 tổng hợp kết quả về chi phí gói tin điều khiển RPL. Với kịch bản tấn công DIS flooding, lý do số lượng gói tin DIS lớn là do các lệnh thu thập số liệu mô phỏng đã thu cả số lượng DIS từ nút tấn công, dẫn đến số lượng DIS rất lớn. Thực tế, số liệu cần xét với kịch bản này là số lượng DIO được gửi đi, do việc gửi DIO chính là phản hồi ngay sau khi nhận được DIS. Có thể thấy việc phòng chống đã làm giảm số lượng DIO gửi đi về mức gần với kịch bản hoạt động bình thường. Đồng thời, số lượng DAO không bị ảnh hưởng nhiều. Điều này cho thấy phương pháp phòng chống hoàn toàn có hiệu

Kịch bản	Tỉ lệ phát hiện đúng	Tỉ lệ phát hiện sai
Phòng chống tấn công DIS flooding	99.98%	0.02%
Phòng chống tấn công phiên bản	95.1%	4.9%
Phòng chống tấn công giảm Rank	100%	0%
Phòng chống tấn công blackhole	100%	0%

Bảng 5.4 Kết quả đánh giá độ chính xác trong việc phát hiện tấn công

quả.

Đối với kịch bản tấn công phiên bản, số lượng gói tin DIO sau khi triển khai biện pháp phòng thủ vẫn cao hơn so với kịch bản hoạt động bình thường, nhưng đã giảm xuống chỉ còn khoảng 43% so với khi bị tấn công mà không có biện pháp phòng thủ. Lý do số gói tin DIS tăng mạnh có thể là do việc gây lỗi phiên bản liên tục khiến các nút không thể hoạt động và thoát khỏi mạng, dẫn đến việc gửi nhiều DIS để yêu cầu tham gia lại. Sự tăng lên của số gói tin DAO trong lúc bị tấn công có thể được quy về việc các nút bị đẩy ra khỏi mạng và sau khi nhận gói tin DIO bất kỳ, sẽ phản hồi lại lên nút gốc bằng gói DAO để yêu cầu tham gia lại. Tuy nhiên, nhóm chưa thể xác định được nguyên nhân cụ thể khiến số gói tin DAO khi được triển khai biện pháp phòng thủ lại thấp hơn đáng kể so với cả kịch bản hoạt động bình thường.

Đối với kịch bản tấn công giảm Rank, số lượng gói tin DIO khi bị tấn công đã tăng lên rất mạnh so với kịch bản hoạt động bình thường, và đã giảm xuống chỉ còn khoảng 1.59 lần so với bình thường sau khi triển khai biện pháp phòng thủ. Tương tự với số gói tin DAO, số gói tin sau khi triển khai biện pháp phòng thủ đã giảm xuống gần với mức bình thường. Tuy nhiên số gói tin DIS vẫn được giữ ở mức ngang bằng với khi bị tấn công. Điều này có thể là do thời gian xử lý tăng lên khi cần phòng thủ, dẫn đến việc một số nút có thể bị đẩy ra khỏi mạng và gửi DIS để yêu cầu tham gia lại.

Đối với kịch bản tấn công blackhole, cả số lượng gói tin DIO và DAO đều tăng lên rất mạnh khi bị tấn công, và đã giảm xuống gần với mức bình thường sau khi triển khai biện pháp phòng thủ. Số lượng gói tin DIS tuy cũng đã được giảm đáng kể, nhưng vẫn còn gấp hơn 2 lần so với kịch bản hoạt động bình thường. Như vậy biện pháp phòng chống đã có hiệu quả về mặt chi phí gói tin điều khiển.

Kịch bản	DIO			DIS			DAO		
	Bình thường	Tấn Công	Phòng vệ	Bình thường	Tấn Công	Phòng vệ	Bình thường	Tấn Công	Phòng vệ
DIS Flooding	166	944	151	0	160783	160789	16	6	10
DAG Version	166	510	220	0	154	27	16	41	3
Decreased Rank	166	18795	264	0	22	29	16	226	12
Blackhole	125	307	151	11	40	25	12	478	10

Bảng 5.5 Tổng số gói tin (DIO / DIS / DAO) theo kịch bản

Bảng 5.6 tổng hợp kết quả về tỉ lệ truyền dữ liệu thành công (Packet Delivery Ratio - PDR) chi tiết theo hướng Upstream và Downstream. Các dạng tấn công DIS và tấn công phiên bản có tỉ lệ truyền dữ liệu thành công gần như không bị ảnh hưởng, trong khi tấn công giảm Rank và tấn công blackhole đã làm giảm đáng kể tỉ lệ này. Việc phòng chống đã giúp cải thiện đáng kể tỉ lệ truyền dữ liệu thành công, đặc biệt là đối với tấn công giảm Rank và tấn công blackhole, đưa chúng trở lại gần với mức bình thường.

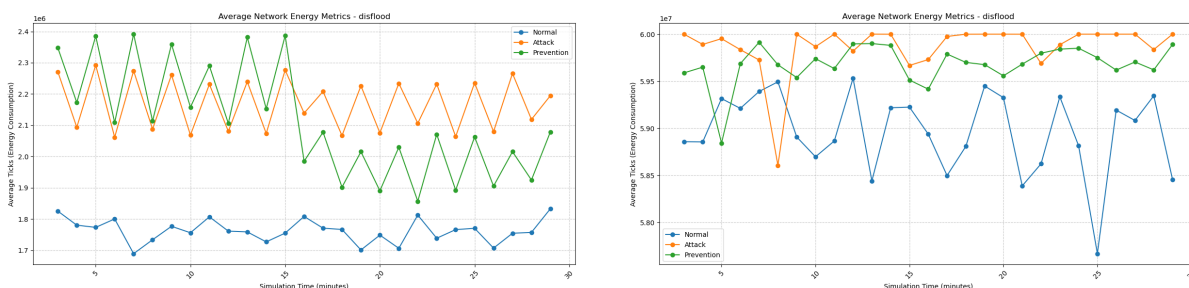
Kịch bản	Hướng	Bình thường	Tấn Công	Phòng vệ
DIS Flooding	Upstream	99.52%	99.82%	99.45%
	Downstream	99.58%	99.38%	99.50%
DAG Version	Upstream	99.52%	99.96%	94.57%
	Downstream	99.58%	99.73%	99.65%
Decreased Rank	Upstream	99.52%	90.28%	99.80%
	Downstream	99.58%	99.66%	99.84%
Blackhole	Upstream	99.95%	0.00%	96.90%
	Downstream	100.00%	0.00%	99.93%

Bảng 5.6 Packet Delivery Ratio (PDR) - Chi tiết Upstream/Downstream

5.5 Phân tích tiêu thụ năng lượng

5.5.1 DIS Flooding

Mức hoạt động của CPU khi được triển khai biện pháp phòng chống nhỉnh hơn so với khi bị tấn công trong nửa đầu của khoảng thời gian đo, nhưng sau đó giảm xuống đáng kể. Bình quân số chu kỳ CPU hoạt động trong mỗi phút giảm từ khoảng 2175681 xuống còn 2117457, nhưng vẫn còn cách xa so với mức bình thường là 1762157. Với số chu kỳ của radio, việc phòng chống đã giảm số này từ 59876622 xuống còn 59678654, nhưng con số này ở gần giá trị khi bị tấn công hơn so với giá trị bình thường (58959914). Như vậy biện pháp phòng chống vẫn chưa tối ưu được mức tiêu thụ năng lượng, dù đã có hiệu quả ở các tiêu chí khác.

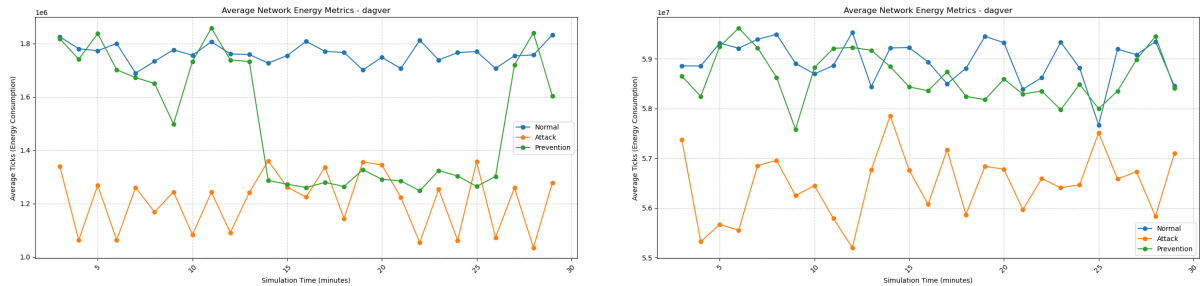


Hình 5.2 Tiêu thụ năng lượng DIS Flooding: CPU (trái) và Radio (phải)

5.5.2 DAG Version Attack

Ngược lại với sự tăng mức tiêu thụ CPU trong kịch bản tấn công DIS, kịch bản tấn công phiên bản lại làm giảm mức tiêu thụ CPU, nhưng lại làm tăng mức tiêu thụ radio. Sự giảm mức tiêu thụ CPU có thể là do việc tấn công phiên bản liên tục khiến các nút bị lỗi phiên bản đồ thị mạng và không thể thực hiện các tác vụ bình thường, dẫn đến việc giảm mức tiêu thụ CPU. Điều tương tự cũng xảy ra với mức tiêu thụ radio. Việc phòng chống nâng mức tiêu thụ CPU từ 1217386 lên 1513039, một khoảng cách đáng kể, và gần với giá trị 1762157 hơn so với khi bị tấn công. Tuy nhiên, nhìn vào đồ thị, lại có thể thấy mức tiêu thụ CPU khi được triển khai biện pháp phòng chống lại có một khoảng sụt xuống gần với mức khi bị tấn công, cụ thể là trong giai đoạn từ 14 đến 26 phút. Với giá trị tiêu thụ Radio, việc phòng chống cho thấy kết quả khả quan hơn, khi đã nâng mức tiêu thụ trung bình từ 56488806 lên 58640745, rất gần với mức bình thường 58959914. Cho thấy phương pháp hash phiên bản đã có hiệu quả trong việc giảm ảnh hưởng đến mức tiêu thụ năng lượng.

Mặc dù một đã có những nghiên cứu cho rằng việc tấn công phiên bản có thể làm tăng mức tiêu thụ năng lượng, kết quả thực nghiệm của nhóm đang cho thấy điều ngược lại. Tuy nhiên, dù mức tiêu thụ năng lượng giảm, ảnh hưởng tiêu cực của tấn công phiên bản, cũng như độ hiệu quả của giải pháp, vẫn rất rõ ràng ở khía cạnh tỉ lệ truyền dữ liệu thành công và chi phí điều khiển RPL.

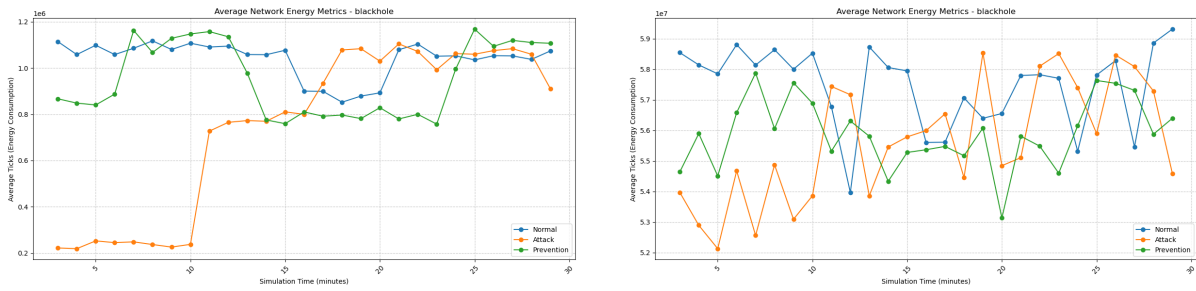


Hình 5.3 Tiêu thụ năng lượng DAG Version Attack: CPU (trái) và Radio (phải)

5.5.3 Blackhole Attack

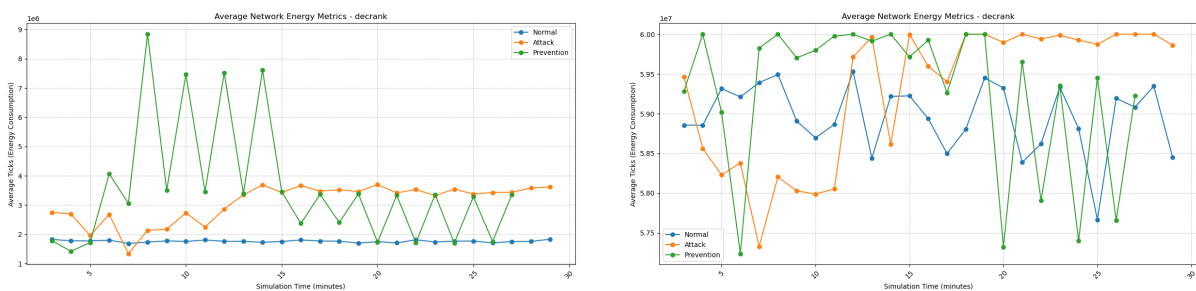
5.5.4 Decreased Rank Attack

Mức tiêu thụ CPU sau khi triển khai biện pháp phòng chống lại tăng gần như đột biến, cho giá trị trung bình 3272003, cao hơn cả giá trị khi bị tấn công (3066664), vốn đã cao hơn gần 2 lần so với mức bình thường (1762156). Mức hoạt động Radio còn cho thấy kết quả khó đánh giá hơn khi liên tục dao động. Giá trị hoạt động trung bình của radio (59446677 chu kỳ/phút) sau khi triển khai biện pháp phòng chống lại còn cao hơn so với khi bị tấn công (59297215 chu kỳ/phút), và cả hai đều cao hơn so với mức bình thường (58959914 chu kỳ/phút). Như vậy, biện pháp phòng chống đã không có hiệu quả



Hình 5.4 Tiêu thụ năng lượng Blackhole Attack: CPU (trái) và Radio (phải)

trong việc giảm mức tiêu thụ năng lượng, mà thậm chí còn làm tăng mức tiêu thụ năng lượng lên đáng kể.



Hình 5.5 Tiêu thụ năng lượng Decreased Rank Attack: CPU (trái) và Radio (phải)

5.6 Tổng kết

Trong chương này, nhóm đã trình bày quá trình thực nghiệm và kết quả đánh giá thu được từ quá trình đó. Các biện pháp phòng chống nhìn chung đã có tác động tích cực đến một số chỉ số (tỉ lệ truyền thành công, chi phí gói tin điều khiển), nhưng lại chưa đủ hiệu quả trong việc giảm mức tiêu thụ năng lượng, thậm chí còn làm tăng mức tiêu thụ năng lượng lên đáng kể trong một số kịch bản. Với thời gian thực nghiệm hạn chế, nhóm chưa thể xác định được nguyên nhân là do bản thân biện pháp phòng chống hay do hướng tiếp cận của nhóm trong việc hiện thực hoá chưa chuẩn xác.

6 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

Trong chương này, nhóm sẽ đưa ra kết luận về những điều đã đạt được trong quá trình thực nghiệm, đồng thời đưa ra những hướng phát triển tiếp theo dựa trên những kết quả đó.

6.1 Kết luận

Nhóm đã thực hiện một loạt các thí nghiệm để đánh giá hiệu quả của các biện pháp phòng chống tấn công DIS, tấn công phiên bản và tấn công giảm rank trong mạng RPL. Kết quả thu được cho thấy rằng các biện pháp phòng chống đã có tác động tích cực đến một số chỉ số quan trọng như tỉ lệ truyền dữ liệu thành công và chi phí gói tin điều khiển. Tuy nhiên, ở khía cạnh tiêu thụ năng lượng, kết quả lại không như mong đợi, khi mức tiêu thụ năng lượng sau khi triển khai biện pháp phòng chống lại còn cao hơn so với khi bị tấn công, và cả hai đều cao hơn so với mức bình thường. Điều này cho thấy rằng mặc dù các biện pháp phòng chống đã có hiệu quả trong việc giảm ảnh hưởng của các cuộc tấn công về mặt truyền dữ liệu và chi phí điều khiển, nhưng vẫn còn nhiều thách thức trong việc tối ưu hóa mức tiêu thụ năng lượng. Do thời gian thực nghiệm hạn chế, nhóm chưa thể xác định được nguyên nhân chính xác của việc tăng mức tiêu thụ năng lượng, có thể là do bản thân biện pháp phòng chống hoặc do cách tiếp cận của nhóm trong việc hiện thực hoá chưa chuẩn xác.

Bên cạnh đó, các biện pháp phòng chống được triển khai đơn lẻ cho từng loại tấn công, chưa có sự kết hợp giữa các biện pháp để tạo ra một hệ thống phòng chống toàn diện hơn. Tuy nhiên, việc triển khai kết hợp nhiều biện pháp có thể không thích hợp với các thiết bị IoT thực tế, vốn bị giới hạn về tài nguyên tính toán và tiêu thụ năng lượng. Do đó, việc tìm kiếm một giải pháp phòng chống hiệu quả và tối ưu hóa mức tiêu thụ năng lượng vẫn là một thách thức lớn đối với nhóm. Mặt khác, do các biện pháp phòng chống được triển khai trên các thiết bị mô phỏng, sự hạn chế và sai lệch trong khả năng mô phỏng quá trình tiêu thụ năng lượng của thiết bị trên Cooja cũng có thể là một yếu tố góp phần vào kết quả không như mong đợi về mức tiêu thụ năng lượng.

6.2 Hướng phát triển

Nếu có thể phát triển tiếp dự án này, nhóm đề xuất một số hướng phát triển sau đây:

- Tìm ra nguyên nhân gốc rễ của việc tăng mức tiêu thụ năng lượng sau khi triển khai biện pháp phòng chống, từ đó có thể tối ưu hóa các biện pháp này để giảm thiểu tác động tiêu cực đến mức tiêu thụ năng lượng.

- Cân nhắc mô phỏng trên các nền tảng sát với thiết bị thật hoặc triển khai trên phần cứng thực tế để có được kết quả chính xác hơn về mức tiêu thụ năng lượng và hiệu quả của các biện pháp phòng chống.
- Nghiên cứu và phát triển các biện pháp phòng chống kết hợp, có thể cân nhắc xây dựng IDS trên thiết bị vật lý riêng, tách biệt với các thiết bị IoT, để giảm tải cho các thiết bị này và tối ưu hóa mức tiêu thụ năng lượng.

TÀI LIỆU THAM KHẢO

- [1] C. Prajisha and A. R. Vasudevan, “Detection and mitigation of multicast DIS flooding attacks in RPL-based IoT networks,” *Ad Hoc Networks*, vol. 179, p. 104002, 2025.
- [2] T. Winter, P. Thubert, A. Brandt, J. Hui, R. Kelsey, P. Levis, K. Pister, R. Struik, J. Vasseur, and R. Alexander, “RPL: IPv6 routing protocol for low-power and lossy networks,” IETF, Tech. Rep. RFC 6550, 2012. [Online]. Available: <https://tools.ietf.org/html/rfc6550>
- [3] A. Dvir, T. Holczer, and L. Buttyán, “VeRA – version number and rank authentication in RPL,” in *2011 IEEE 8th International Conference on Mobile Ad-Hoc and Sensor Systems (MASS)*, 2011, pp. 709–714.
- [4] LINGI2146 Project – UCLouvain, “RPL attacks framework,” 2016, course project report.
- [5] D. K. Sharma, S. K. Dhurandher, S. Kumaram, K. Datta Gupta, and P. K. Sharma, “Mitigation of black hole attacks in 6LoWPAN RPL-based wireless sensor network for cyber physical systems,” *Computer Communications*, vol. 189, pp. 182–192, 2022.
- [6] S. Raza, L. Wallgren, and T. Voigt, “SVELTE: Real-time intrusion detection in the internet of things,” *Ad Hoc Networks*, vol. 11, no. 8, pp. 2661–2674, 2013.
- [7] L. Wallgren, S. Raza, and T. Voigt, “Routing attacks and countermeasures in the RPL-based internet of things,” in *International Journal of Distributed Sensor Networks*, vol. 9, no. 8, 2013.

PHỤ LỤC

A Mã nguồn IDS (Contiki-NG)

A.1 Module phát hiện DIS Flooding

```
1 #include "contiki.h"
2 #include "net/routing/rpl-lite/rpl.h"
3
4 /* Nguong so ban tin DIS trong mot chu ky Trickle */
5 #define DIS_FLOOD_THRESHOLD 10
6
7 static uint16_t dis_count = 0;
8
9 void ids_dis_on_receive(void) {
10     dis_count++;
11     if (dis_count > DIS_FLOOD_THRESHOLD) {
12         /* TODO: Kich hoat canh bao DIS Flooding */
13     }
14 }
```

Listing 1: Skeleton: ids_dis_flooding.c

A.2 Module phát hiện Blackhole

```
1 #include "contiki.h"
2
3 #define PDR_THRESHOLD 0.7 /* PDR duoi 70% -> nghi ngo
4     Blackhole */
5
6 float ids_compute_pdr(uint16_t sent, uint16_t received) {
7     if (sent == 0) return 1.0f;
8     return (float)received / (float)sent;
9 }
```

Listing 2: Skeleton: ids_blackhole.c

A.3 Module phát hiện Decreased Rank

```
1 #include "contiki.h"
2 #include "net/routing/rpl-lite/rpl.h"
```



```

3
4 void ids_rank_check(rpl_dag_t *dag, rpl_parent_t *parent,
5                     rpl_rank_t advertised_rank) {
6     rpl_rank_t expected = /* tinh toan rank du kien */ 0;
7     if (advertised_rank < expected) {
8         /* TODO: Danh dau parent nay la nghi ngo */
9     }
10 }

```

Listing 3: Skeleton: ids_decreased_rank.c

A.4 Module phát hiện VeRA / Version Number Attack

```

1 #include "contiki.h"
2 #include "net/routing/rpl-lite/rpl.h"
3
4 static uint8_t last_known_version = 0;
5
6 void ids_version_check(uint8_t received_version) {
7     if (received_version > last_known_version + 1) {
8         /* TODO: Version tang bat thuong -> canh bao VeRA attack
9          */
10    }
11    last_known_version = received_version;
12 }

```

Listing 4: Skeleton: ids_vera.c

B Cấu hình mô phỏng Cooja

B.1 Tham số mô phỏng

Bảng B.7 Tham số cấu hình chung trong Cooja

Tham số	Giá trị
Mô phỏng radio	Unit Disk Graph Medium (UDGM)
Số nút tổng	20 nút (1 Border Router + 19 mote)
Tỉ lệ nút tấn công	0%, 10%, 25%, 50%
Thời gian mô phỏng	300 giây / kịch bản
Tần suất gửi dữ liệu	1 gói / 10 giây
Hệ điều hành	Contiki-NG 4.x
Objective Function	MRHOF (ETX metric)

B.2 Hướng dẫn chạy mô phỏng

1. Cài đặt Contiki-NG và Cooja theo hướng dẫn tại <https://github.com/contiki-ng/contiki-ng/wiki>.
2. Mở file `simulation/rpl_ids_scenario.csc` trong Cooja.
3. Chọn kịch bản tấn công trong menu **Simulation Parameters**.
4. Nhấn **Start** và quan sát log từ **Mote Output**.
5. Xuất kết quả ra file `.csv` để phân tích với script Python trong thư mục `analysis/`.